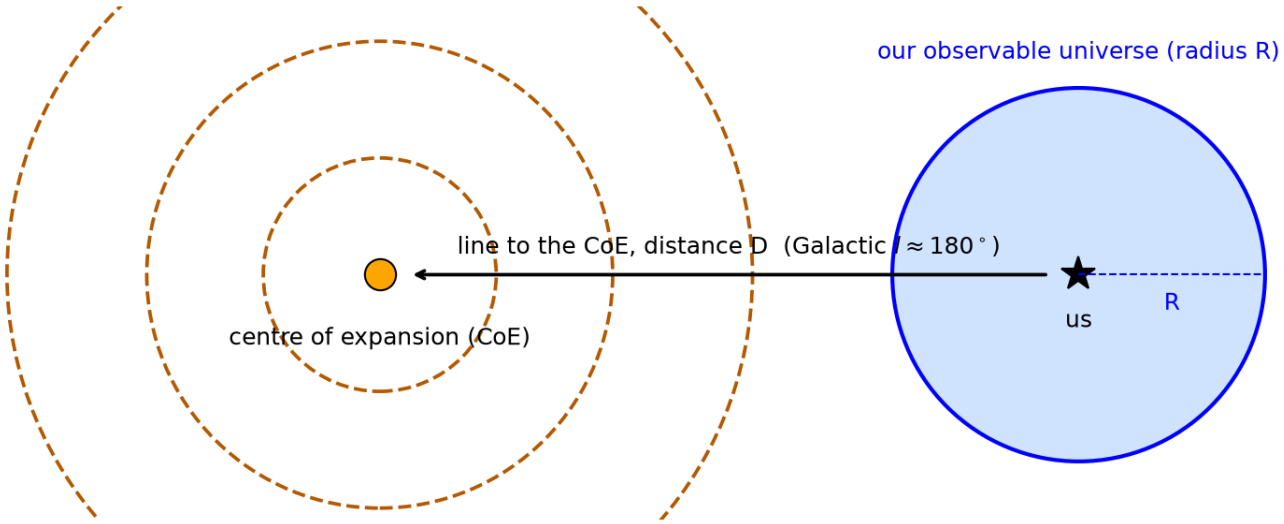
Python scripts for every figure in CoE_letter_2026-10-27.pdf

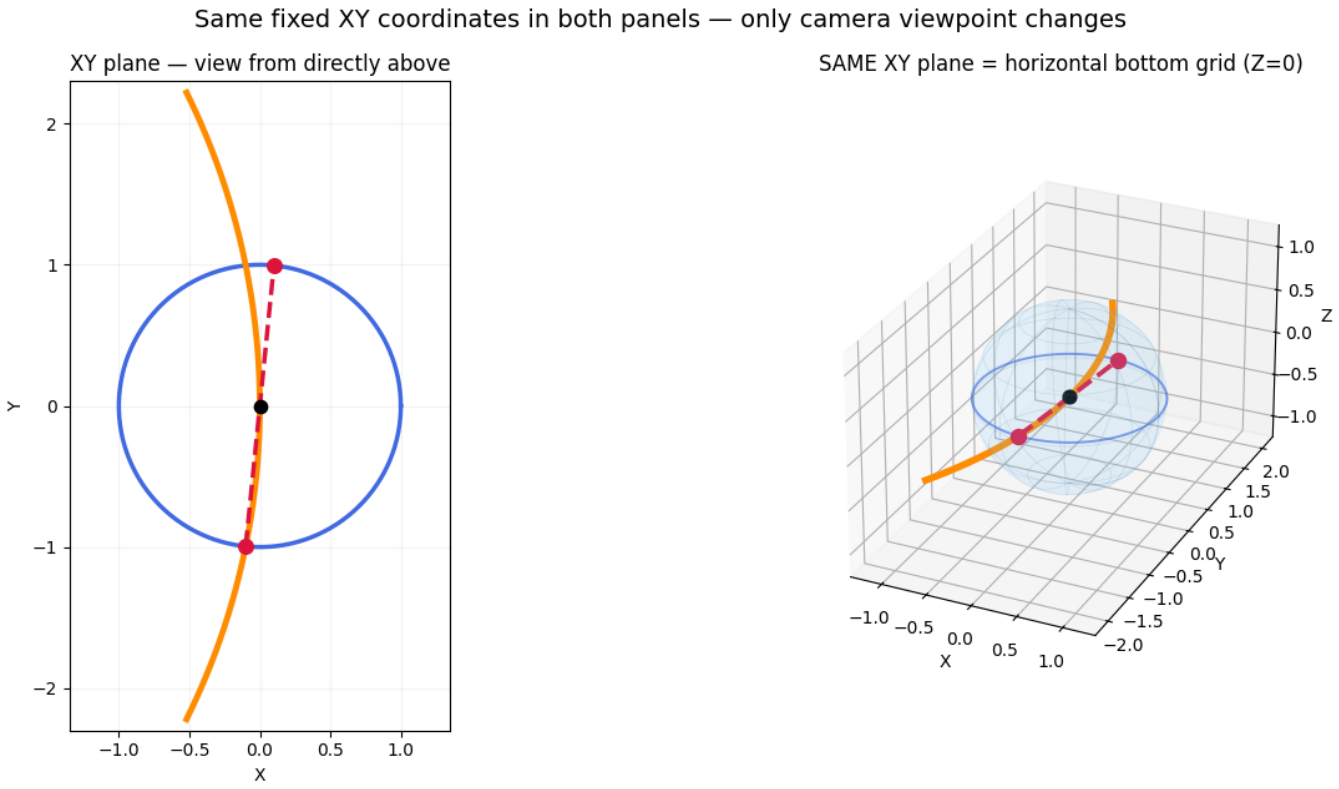
Figures listed in the order they appear in the letter.

Figure 1: **geometry_schematic.png — geometry_schematic.py**



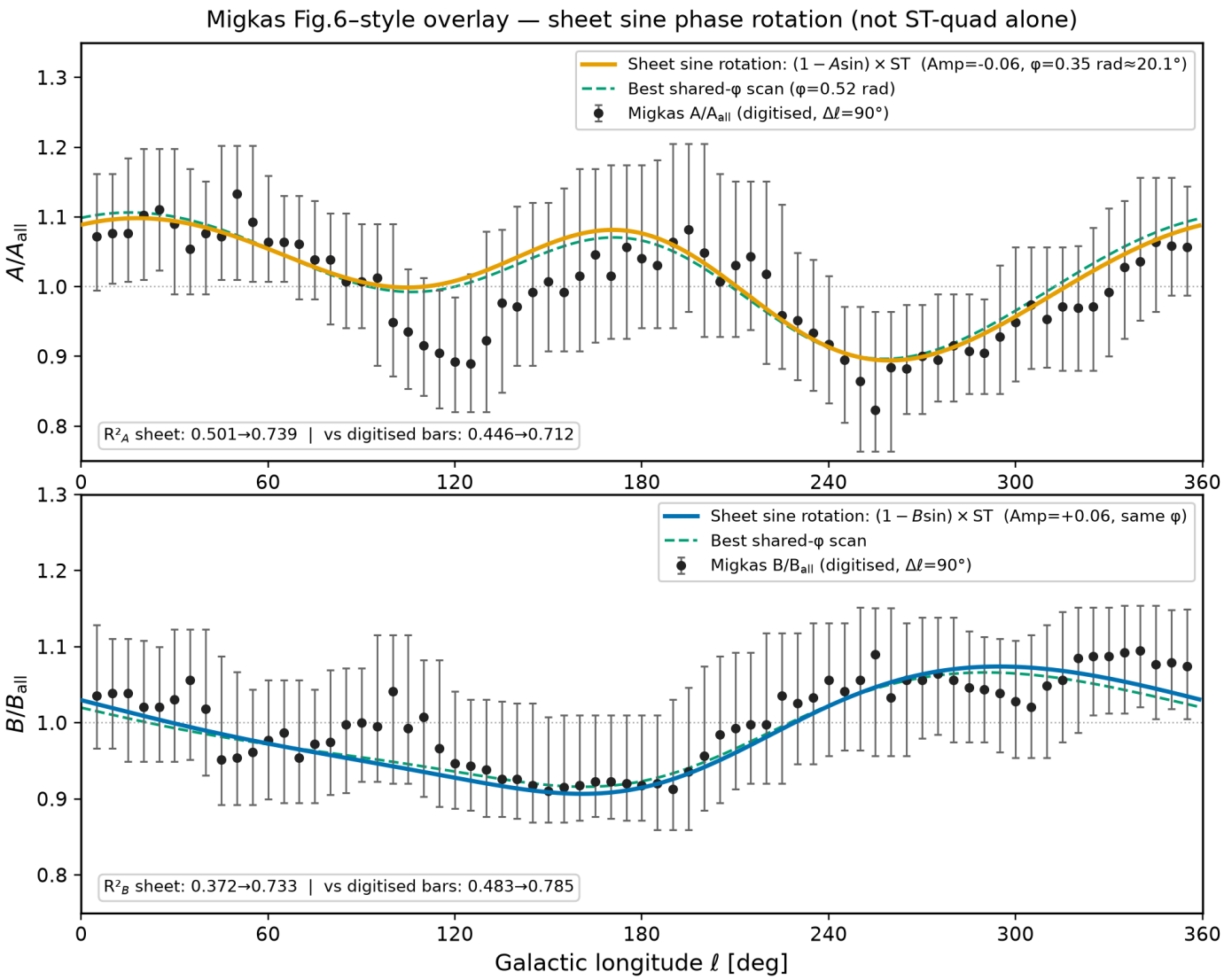
```
# Simple schematic: CoE, observer offset by D, observable universe as a circle around the observer.
import os,numpy as np,matplotlib;matplotlib.use('Agg');import matplotlib.pyplot as plt
H=os.path.dirname(os.path.abspath(__file__))
fig,ax=plt.subplots(figsize=(10,4.2))
D,R=6.0,1.6
ax.add_patch(plt.Circle((D,0),R,fc='#cfe3ff',ec='b',lw=2))
ax.text(D,R+0.25,'our observable universe (radius R)',ha='center',color='b',fontsize=12)
ax.plot(0,0,'o',color='orange',ms=16,mec='k');ax.text(0,-0.6,'centre of expansion (CoE)',ha='center',fontsize=12)
for r in (1,2,3.2):ax.add_patch(plt.Circle((0,0),r,fill=False,ec='#b35900',ls='--',lw=1.8))
ax.plot(D,0,'k*',ms=18);ax.text(D,-0.45,'us',ha='center',fontsize=12)
ax.annotate('',(0.25,0),(D-0.25,0),arrowprops=dict(arrowstyle='->',lw=1.8))
ax.text(D/2,0.18,'line to the CoE, distance D (Galactic $l\\approx180^\\circ$)',ha='center',fontsize=12)
ax.plot([D,D+R],[0,0],'b--',lw=1);ax.text(D+R/2,-0.3,'R',color='b',ha='center',fontsize=12)
ax.set_aspect('equal');ax.set_xlim(-3.5,D+R+0.5);ax.set_ylim(-2.1,2.3);ax.axis('off')
fig.tight_layout();fig.savefig(H+'/geometry_schematic.png',dpi=150,facecolor='white')
```

Figure 2: **chris_equal_distance_arc.png — chris_equal_distance_arc_OLD.py**



```
# Equal-distance arc: points as far from the CoE as the observer. Observer at origin, R=1 sphere,
# CoE at (-D,0) (Galactic l=180 along -X), D=2.5. Arc = circle radius D about CoE through observer.
import os,numpy as np,matplotlib;matplotlib.use('Agg');import matplotlib.pyplot as plt
H=os.path.dirname(os.path.abspath(__file__));D=2.5
# intersection of |p|=1 and |p-(-D,0)|=D: x=-1/(2D)
x=-1/(2*D);y=np.sqrt(1-x*x);P1,P2=(x,y),(x,-y)
lP1=np.degrees(np.arctan2(y,x))%360;lP2=np.degrees(np.arctan2(-y,x))%360
t=np.linspace(0,2*np.pi,400);ph=np.radians(np.linspace(-40,40,200))
ax_,ay_=-D+D*np.cos(ph),D*np.sin(ph)
fig=plt.figure(figsize=(15,7))
a=fig.add_subplot(121)
a.plot(np.cos(t),np.sin(t),'b',lw=2,label='unit sphere R=1')
a.plot(ax_,ay_,color='orange',lw=3,label=f'arc: equal distance D={D} from CoE')
a.plot([P1[0],P2[0]],[P1[1],P2[1]],'r--',lw=1.5,label='chord —P1P2')
a.plot(*zip(P1,P2),'ro',ms=10)
for p,n,l in [(P1,'P1',lP1),(P2,'P2',lP2)]:a.annotate(f'{n} l={l:.1f}°',p,xytext=(10,10 if p[1]>0 else -20),textcoords='offset points',color='r',fontsize
    =12)
a.plot(0,0,'k*',ms=16,label='observer');a.annotate('-X →l=180° (CoE)',(-1.45,0.05),fontsize=10)
a.plot([-1.6,0],[0,0],'k:',lw=1)
a.set_aspect('equal');a.set_xlim(-1.7,1.4);a.set_ylim(-1.4,1.4);a.set_xlabel('X');a.set_ylabel('Y');a.grid(alpha=.3)
a.set_title(f'Top-down XY plane (CoE at −({D},0), off panel)');a.legend(loc='lower right',fontsize=9)
b=fig.add_subplot(122,projection='3d')
u,v=np.meshgrid(np.linspace(0,2*np.pi,40),np.linspace(0,np.pi,20))
b.plot_surface(np.cos(u)*np.sin(v),np.sin(u)*np.sin(v),np.cos(v),color='b',alpha=.12,linewidth=0)
g=np.linspace(-1.5,1.5,7)
for c in g:b.plot([c,c],[-1.5,1.5],0,color='0.7',lw=.6);b.plot([-1.5,1.5],[c,c],0,color='0.7',lw=.6)
b.plot(np.cos(t),np.sin(t),0,'b',lw=2);b.plot(ax_,ay_,0,color='orange',lw=3)
b.plot([P1[0],P2[0]],[P1[1],P2[1]],[0,0],'r--');b.scatter(*zip(P1,P2),[0,0],c='r',s=50,depthshade=False)
for p,n in [(P1,'P1'),(P2,'P2')]:b.text(p[0],p[1],0.1,n,color='r')
b.scatter([0],[0],[0],c='k',marker='*',s=150)
b.set_xlim(-1.5,1.5);b.set_ylim(-1.5,1.5);b.set_zlim(-1.5,1.5);b.set_box_aspect((1,1,1))
b.set_xlabel('X');b.set_ylabel('Y');b.set_zlabel('Z');b.view_init(25,-60);b.set_title('3D view, Z=0 grid')
fig.suptitle('Same fixed XY coordinates in both panels —only camera viewpoint changes',fontsize=14)
fig.tight_layout();fig.savefig(H+'/chris_equal_distance_arc.png',dpi=120,facecolor='white');print(lP1,lP2)
```

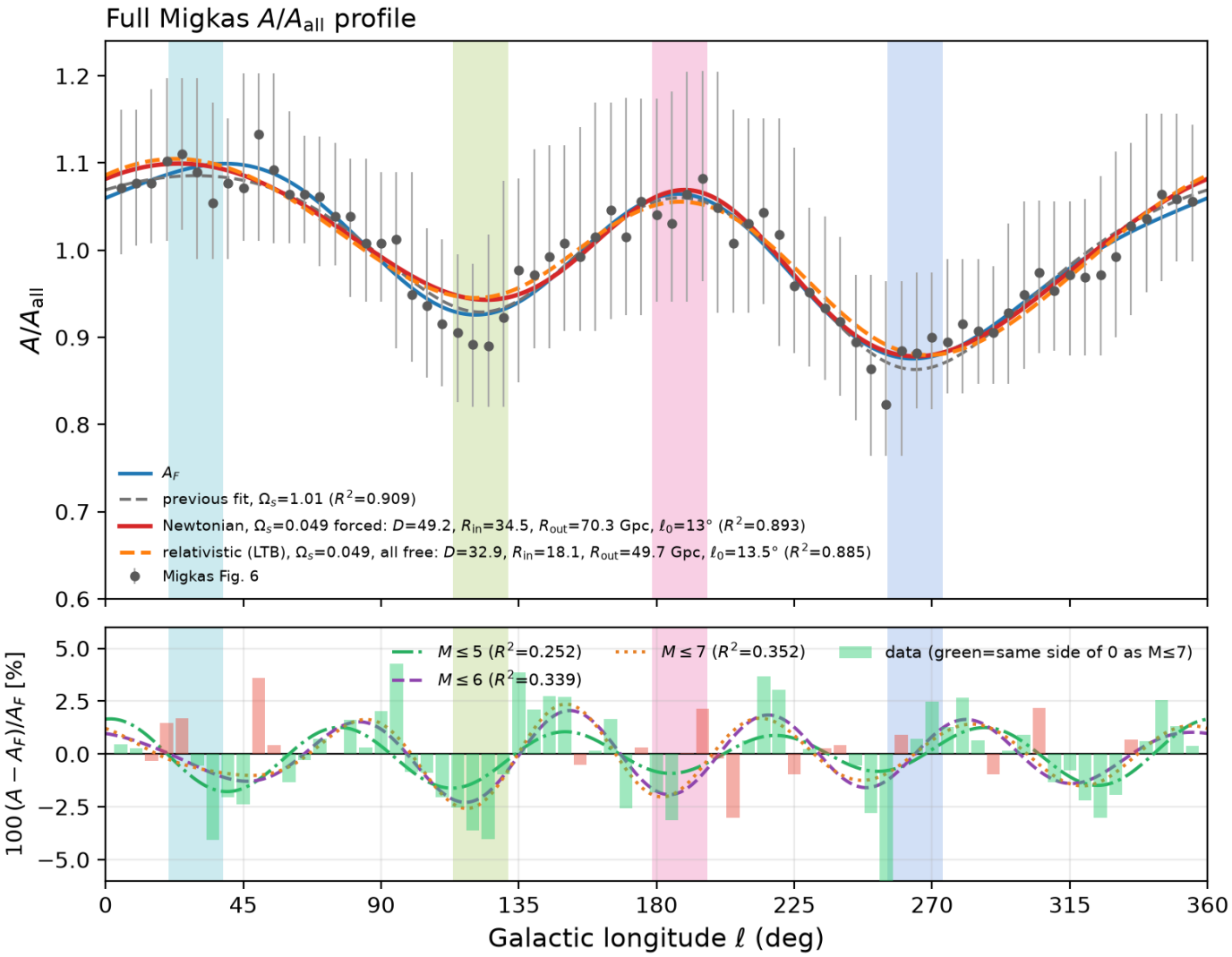
Figure 3: `R04_migkas_rotation.png` — `R04_migkas_rotation.py`



Diagnostic: Amp×sin((2π/360)·ℓ+φ) phase alignment from Chris sheet tab "Copy of Copy of Copy of density". ST quadrupole alone is a separate claim.

```
# Generates R04_migkas_rotation.png. Real code: rerun/L04_migkas_rotation.py (run from this folder).
import runpy,os
os.chdir(os.path.dirname(os.path.abspath(__file__)))
runpy.run_path("rerun/L04_migkas_rotation.py",run_name="__main__")
```

Figure 4: **Fig04_amp_full_stripes.png** — **Fig04_amp_full_stripes.py**



```python
#!/usr/bin/env python3
"""Fig04 —Full Migkas A/Aall + multipole residual fits (NO Amp×sin squiggles).

Top: Migkas + A_F + previous/Newt/Rel curves (letter gold style).
Bottom: cumulative Fourier fits ≤—M57 to —MigkasA_F residuals (same as Fig02).
Forbidden asset: money_shot_amp_full_SINE_BAD.png (do not use).
"""
from __future__ import annotations

import sys
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np

HERE = Path(__file__).resolve().parent
sys.path.insert(0, str(HERE))
from paths import OUT # noqa: E402
from physics import ( # noqa: E402
    A_F,
    BAND_CENTRES,
    BAND_COLORS,
    BAND_HALF,
    load_fit_curves,
    load_migkas_fig6,
    r2_score,
)

OUT_NAME = "Fig04_amp_full.png"


def design(ell_deg, M):
    """Cumulative multipoles through M: a0 + sum_{m=1..M} [c_m cos(□m)+s_m sin(□m)]."""
    rad = np.radians(np.asarray(ell_deg, float))
    cols = [np.ones_like(rad)]
    for m in range(1, M + 1):
        cols.append(np.cos(m * rad))
        cols.append(np.sin(m * rad))
    return np.column_stack(cols)


def fit_residual_M(ell, r, er, M):
    X = design(ell, M)
    ww = 1.0 / np.maximum(er**2, 1e-18)
    XtW = X.T * ww
    beta = np.linalg.lstsq(XtW @ X, XtW @ r, rcond=None)[0]
    yhat = X @ beta
    ybar = np.sum(ww * r) / np.sum(ww)
    ss_res = np.sum(ww * (r - yhat) ** 2)
    ss_tot = np.sum(ww * (r - ybar) ** 2)
    R2 = 1.0 - ss_res / ss_tot if ss_tot > 0 else np.nan
    return beta, float(R2)


def main():
    OUT.mkdir(parents=True, exist_ok=True)
    ell_d, A_d, e_lo, e_hi = load_migkas_fig6()
    th, A_newt, A_free, A_rel, meta = load_fit_curves()
    mn, mf, mr = meta["newt"], meta["free"], meta["rel"]
    th2 = np.append(th, 360.0)
    cl = lambda y: np.append(y, y[0])

    AF_pts = A_F(ell_d)
    r = 100.0 * (A_d - AF_pts) / AF_pts
    er_lo = 100.0 * e_lo / AF_pts
    er_hi = 100.0 * e_hi / AF_pts
    er = 0.5 * (er_lo + er_hi) # symmetric weight for WLS

    R2f = float(mf["R2"])
    R2n = float(mn["R2"])
    R2r = float(mr["R2"])

    # Letter gold ~1800×1385 →figsize ~9×6.9 at 200dpi
    fig, (a, b) = plt.subplots(
        2, 1, figsize=(9.0, 6.9), sharex=True,
        gridspec_kw={"height_ratios": [2.2, 1.0], "hspace": 0.07},
    )
    for L, col in zip(BAND_CENTRES, BAND_COLORS):
        for ax in (a, b):
            ax.axvspan(L - BAND_HALF, L + BAND_HALF, color=col, alpha=0.20, lw=0, zorder=0)

    a.errorbar(
        ell_d, A_d, yerr=[e_lo, e_hi], fmt="o", ms=3.5, color="0.35", ecolor="0.65",
        lw=0.8, zorder=3, label="Migkas Fig. 6",
    )
    a.plot(th2, A_F(th2), color="C0", lw=1.7, label=r"$A_F$", zorder=2)
    a.plot(
        th2, cl(A_free), color="0.45", ls="--", lw=1.4,
```

```
        label=rf"previous fit, $\Omega_s$={mf['Os']:.2f} ($R^2$={R2f:.3f})",
    )
    a.plot(
        th2, cl(A_newt), color="C3", lw=2.0,
        label=(
            rf"Newtonian, $\Omega_s$={mn['Os']:.3f} forced: "
            rf"$D$={mn['D']:.1f}, $R_{{\rm in}}$={mn['R_in']:.1f}, "
            rf"$R_{{\rm out}}$={mn['R_out']:.1f} Gpc, "
            rf"$\ell_0$={mn['l0']:.0f}° ($R^2$={R2n:.3f})"
        ),
    )
    a.plot(
        th2, cl(A_rel), color="C1", lw=1.8, ls="--",
        label=(
            rf"relativistic (LTB), $\Omega_s$={mr['Os']:.3f}, all free: "
            rf"$D$={mr['D']:.1f}, $R_{{\rm in}}$={mr['R_in']:.1f}, "
            rf"$R_{{\rm out}}$={mr['R_out']:.1f} Gpc, "
            rf"$\ell_0$={mr['l0']:.1f}° ($R^2$={R2r:.3f})"
        ),
    )
    a.set_ylabel(r"$A/A_{\mathrm{all}}$", fontsize=12)
    a.set_ylim(0.60, 1.24)
    a.legend(fontsize=7.0, loc="lower left", frameon=False)
    a.set_title(r"Full Migkas $A/A_{\mathrm{all}}$ profile", loc="left", fontsize=12)

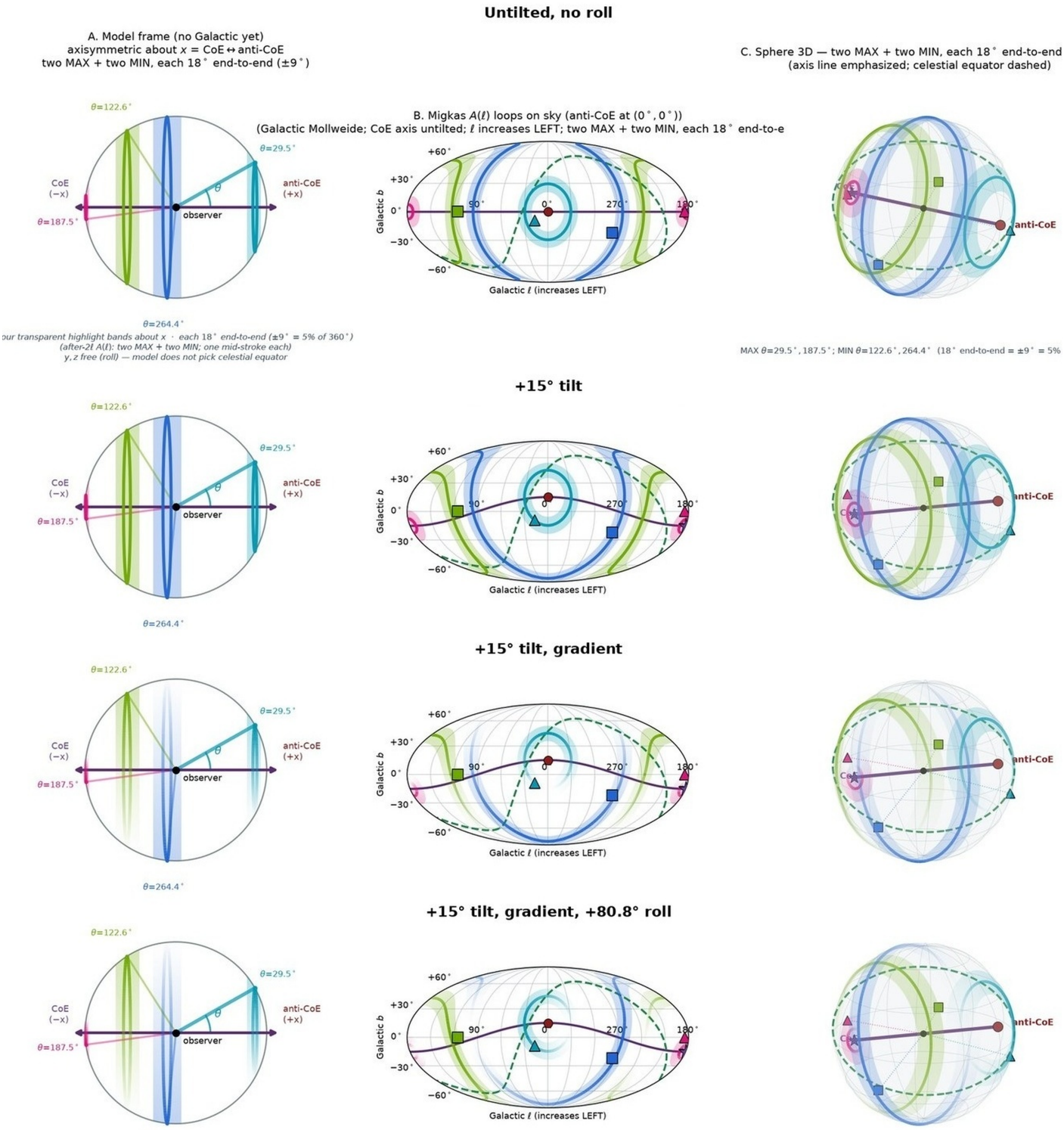
    # Bottom: cumulative ≤M57 residual fits (same as Fig02)
    b.axhline(0, color="k", lw=0.8)
    # bars: green if residual same side of 0 as M<=7, else red
    order = np.argsort(ell_d)
    ell_o, r_o = ell_d[order], r[order]
    beta7, _ = fit_residual_M(ell_d, r, er, 7)
    y7 = design(ell_o, 7) @ beta7
    same_side = (r_o * y7) > 0
    gaps = np.diff(ell_o)
    widths = np.empty_like(r_o)
    widths[:-1] = np.maximum(gaps * 0.85, 1.5)
    widths[-1] = widths[-2] if len(widths) > 1 else 3.0
    colors = np.where(same_side, "#2ecc71", "#e74c3c")
    # zero residual or zero model: treat as red (not same side)
    colors = np.where(same_side, "#2ecc71", "#e74c3c")
    b.bar(
        ell_o, r_o, width=widths, align="center",
        color=colors, alpha=0.45, edgecolor="none", zorder=3,
        label="data (green=same side of 0 as ≤M7)",
    )

    ell_g = np.linspace(0, 360, 721)
    colors = {5: "#27ae60", 6: "#8e44ad", 7: "#e67e22", 8: "#2980b9"}
    lss = {5: "-.", 6: "--", 7: ":", 8: "-"}
    lws = {5: 1.5, 6: 1.5, 7: 1.5, 8: 1.7}
    for M in (5, 6, 7):
        beta, R2 = fit_residual_M(ell_d, r, er, M)
        b.plot(
            ell_g, design(ell_g, M) @ beta,
            color=colors[M], ls=lss[M], lw=lws[M],
            label=rf"$M\leq{M}$ ($R^2$={R2:.3f})",
        )
    b.set_ylim(-6, 6) # keep residual scale; do not autoscale to large Migkas error whiskers
    b.set_ylabel(r"$100\,(A-A_F)/A_F$ [%]", fontsize=10)
    b.set_xlabel(r"Galactic longitude $\ell$ (deg)", fontsize=12)
    b.set_xlim(0, 360)
    b.set_xticks(range(0, 361, 45))
    b.legend(fontsize=7.5, frameon=False, loc="upper right", ncol=3)
    b.grid(True, alpha=0.25)

    dest = OUT / OUT_NAME
    fig.savefig(dest, dpi=200, bbox_inches="tight", facecolor="white")
    plt.close(fig)
    print("wrote", dest, dest.stat().st_size)
    return dest


if __name__ == "__main__":
    main()
```

Figure 5: `fig8_orientation_seq.png` — `make_migkas_fig8_overlay_matched.py`



```
#!/usr/bin/env python3
"""
Superimpose CoE MAX/MIN highlight bands + Migkas markers onto the REAL Migkas
Fig. 8 theta=75 A/A_all Mollweide map.

Matches make_coe_rings_migkas_pin_axis.py (three-panel Panel B) geometry:
  Exactly FOUR features —each ONE transparent highlight band, 18° end-to-end
  (= ±9° = 5% of 360°) about the center θ+ ONE solid mid-stroke:
    MAX 30° →[21°, 39°] color #00e5ff (bright cyan)
    MAX 185° →[176°, 194°] color #ff2d95 (magenta/hot pink)
    MIN 115° →[106°, 124°] color #b8f200 (lime)
    MIN 265° →[256°, 274°] color #4fc3f7 (sky blue)
  Drawing: clean PolyCollection fill + one mid-stroke (optional thin white
  underlay on mid only). NO black rim, NO thick white halo stacks, NO dual
  boundary-edge strokes. No 283° companion. No Dec=0 overlay.
  MAX triangles: (17°, −9°) and (185°, 0°)
  MIN squares: (115°, 0°) and (274°, −22°)
  CoE axis tilted +30°: anti-CoE (0°,+30°), CoE (180°−,30°) (purple).

Registration (unchanged from working Fig. 8):
  GOOD_REG = (cx, cy, Rx, Ry) = (863.0, 763.25, 693.9927, 392.542425)
  Source: FULL_migkas_theta75.png
  CoE axis tilted +30°; clip/inset so overlays stay inside colored oval
  lon_disp = -wrap□()

Outputs:
  migkas_fig8_theta75_with_coe_rings.png
  migkas_fig8_theta75_with_coe_rings.json
"""

from __future__ import annotations

import json
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
from matplotlib.collections import PolyCollection
from matplotlib.lines import Line2D
from matplotlib.patches import Ellipse, Patch
from PIL import Image

OUT_DIR = Path(__file__).resolve().parent
SM_DIR = Path("/workspace/cosmology/papers/spreadsheet_model")
PNG_PATH = SM_DIR / "fig8_overlay_matched.png"
JSON_PATH = SM_DIR / "fig8_overlay_matched.json"

# Versions = panel B of each three-panel figure (grad: opacity gradient, roll: deg CCW about CoE axis)
VERSIONS = [
   ("matched", False, 0.0),
   ("matched_grad", True, 0.0),
   ("matched_grad_rot45", True, 45.0),
   ("matched_grad_rot81", True, 80.8),
   ("matched_rot81", False, 80.8), # roll + all loops, no opacity gradient
]
GRAD = False
ROLL = 0.0
SHOW_LEGEND = False # letter Fig.6: no legend box/strip

SOURCE_CANDIDATES = [
   Path("/workspace/cosmology/map_compare/FULL_migkas_theta75.png"),
   Path("/workspace/cosmology/map_compare/migkas_fig8_theta75_crop.png"),
   Path("/workspace/cosmology/migkas/pub_maps/Fig8_full_hires.png"),
]

# Strip A□() scan stops from +x (after□-2 A_after extrema) —four band centers.
# 5% of 360° = 18° end-to-end →half-width ±9° about each center (match three-panel).
BAND_HALF = 9.0
BAND_FULL = 2.0 * BAND_HALF # 18°
THETA_MAX = [29.5, 187.5]
THETA_MIN = [122.6, 264.4]
MAX_BANDS = [(t - BAND_HALF, t + BAND_HALF) for t in THETA_MAX] # [21,39], [176,194]
MIN_BANDS = [(t - BAND_HALF, t + BAND_HALF) for t in THETA_MIN] # [106,124], [256,274]


def mu_pair_for_band(lo, hi):
   """Edge μ=cosθ() for a scan-angle band μ(-space, not θ-space)."""
   return float(np.cos(np.deg2rad(lo))), float(np.cos(np.deg2rad(hi)))

# Visually-good registration (pre medial-axis retune). Hard-set as working default.
GOOD_REG = {
   "cx": 863.0,
   "cy": 763.25,
   "Rx": 693.9927,
   "Ry": 392.542425,
}

# Sky markers □(, b): map extrema + strip stand-ins (same as 3-panel Panel B).
```

```
MIGKAS_MAXES = [(17.0, -9.0), (185.0, 0.0)]
MIGKAS_MINS = [(115.0, 0.0), (274.0, -22.0)]

# Galactic pose (Chris 2026-09-22): anti-CoE (+D/+x) at (0°, +30°); CoE at (180°, −30°)
ANTI_COE_LB = (0.0, 15.0)
COE_LB = (180.0, -15.0)
AXIS_TILT_B_DEG = 15.0


def lb_to_n(l_deg, b_deg):
    l = np.deg2rad(l_deg)
    b = np.deg2rad(b_deg)
    return np.array(
        [np.cos(b) * np.cos(l), np.cos(b) * np.sin(l), np.sin(b)], dtype=float
    )


D_HAT = lb_to_n(*ANTI_COE_LB) # +D = anti-CoE = +x


def great_circle_through_poles(D, n=721):
    """Great circle through ±D (CoE axis meridian on the sky)."""
    a = D / np.linalg.norm(D)
    tmp = np.array([0.0, 0.0, 1.0])
    if abs(np.dot(a, tmp)) > 0.9:
        tmp = np.array([1.0, 0.0, 0.0])
    e2 = np.cross(a, tmp)
    e2 /= np.linalg.norm(e2)
    t = np.linspace(0.0, 2.0 * np.pi, n)
    pts = np.outer(np.cos(t), a) + np.outer(np.sin(t), e2)
    ls = np.array([n_to_lb(p)[0] for p in pts])
    bs = np.array([n_to_lb(p)[1] for p in pts])
    return ls, bs

# High-contrast loop colors that pop on magma (avoid reds/oranges/corals)
C_MAX_30 = "#0097b2" # cyan —MAX 29.5° (three-panel matched)
C_MAX_185 = "#e0107a" # magenta —MAX 187.5°
C_MIN_115 = "#6fa800" # green —MIN 122.6°
C_MIN_265 = "#2166d4" # blue —MIN 264.4°
COLORS_MAX = [C_MAX_30, C_MAX_185]
COLORS_MIN = [C_MIN_115, C_MIN_265]
C_COE = "#6b3fa0"
C_ANTI = "#8e1b1b" # dark red marker; distinct from loop reds (match three-panel)
C_AXIS = "#5b2c6f"
C_EQ = "#1e8449"


def wrap_lon_deg(lon):
    return ((np.asarray(lon, dtype=float) + 180.0) % 360.0) - 180.0


def lon_disp_from_ell(ell_deg):
    return -wrap_lon_deg(ell_deg)


def n_to_lb(n):
    n = np.asarray(n, dtype=float)
    n = n / np.linalg.norm(n)
    b = float(np.rad2deg(np.arcsin(np.clip(n[2], -1.0, 1.0))))
    l = float(np.rad2deg(np.arctan2(n[1], n[0])) % 360.0)
    return l, b


def celestial_equator_lb(n=2500):
    ra_ngp = np.deg2rad(192.85948)
    dec_ngp = np.deg2rad(27.12825)
    l_ncp = np.deg2rad(122.93192)
    ra = np.linspace(0.0, 2.0 * np.pi, n, endpoint=False)
    dec = 0.0
    sin_b = np.clip(
        np.sin(dec) * np.sin(dec_ngp)
        + np.cos(dec) * np.cos(dec_ngp) * np.cos(ra - ra_ngp),
        -1.0,
        1.0,
    )
    b = np.arcsin(sin_b)
    y = np.cos(dec) * np.sin(ra - ra_ngp)
    x = (
        np.sin(dec) * np.cos(dec_ngp)
        - np.cos(dec) * np.sin(dec_ngp) * np.cos(ra - ra_ngp)
    )
    l = (l_ncp - np.arctan2(y, x)) % (2.0 * np.pi)
    return np.rad2deg(l), np.rad2deg(b)


def small_circle_about_axis(axis, mu, n=721):
    a = axis / np.linalg.norm(axis)
    mu = float(np.clip(mu, -1.0 + 1e-9, 1.0 - 1e-9))
```

```
    tmp = np.array([0.0, 0.0, 1.0]) if abs(a[2]) < 0.9 else np.array([1.0, 0.0, 0.0])
    e1 = np.cross(a, tmp)
    e1 /= np.linalg.norm(e1)
    e2 = np.cross(a, e1)
    rho = np.sqrt(max(0.0, 1.0 - mu * mu))
    t = np.linspace(0.0, 2.0 * np.pi, n)
    return (mu * a)[None, :] + rho * (
       np.outer(np.cos(t), e1) + np.outer(np.sin(t), e2)
    )


def mollweide_xy(lon_rad, lat_rad, n_iter=64):
    """Standard Mollweide; stable bracketed solve of θ2+sinθ(2)π= sin(lat)."""
    lon = np.asarray(lon_rad, dtype=float)
    lat = np.asarray(lat_rad, dtype=float)
    target = np.pi * np.sin(np.clip(lat, -np.pi / 2, np.pi / 2))
    lo = np.full_like(lat, -np.pi / 2, dtype=float)
    hi = np.full_like(lat, np.pi / 2, dtype=float)
    for _ in range(n_iter):
       mid = 0.5 * (lo + hi)
       g = 2.0 * mid + np.sin(2.0 * mid)
       lo = np.where(g < target, mid, lo)
       hi = np.where(g >= target, mid, hi)
    theta = 0.5 * (lo + hi)
    theta = np.where(lat >= np.pi / 2 - 1e-15, np.pi / 2, theta)
    theta = np.where(lat <= -np.pi / 2 + 1e-15, -np.pi / 2, theta)
    x = (2.0 * np.sqrt(2.0) / np.pi) * lon * np.cos(theta)
    y = np.sqrt(2.0) * np.sin(theta)
    return x, y


def lb_to_px(ell_deg, b_deg, cx, cy, Rx, Ry):
    lon = np.deg2rad(lon_disp_from_ell(np.asarray(ell_deg, dtype=float)))
    lat = np.deg2rad(np.asarray(b_deg, dtype=float))
    x, y = mollweide_xy(lon, lat)
    Xn = x / (2.0 * np.sqrt(2.0))
    Yn = y / np.sqrt(2.0)
    return cx + Xn * Rx, cy - Yn * Ry


def as_xy(px, py):
    return float(np.asarray(px).reshape(-1)[0]), float(np.asarray(py).reshape(-1)[0])


def chamfer_distance_to_dark(dark):
    h, w = dark.shape
    INF = 1e6
    D = np.where(dark, 0.0, INF)
    for y in range(h):
       for x in range(w):
          v = D[y, x]
          if x > 0:
             v = min(v, D[y, x - 1] + 1.0)
          if y > 0:
             v = min(v, D[y - 1, x] + 1.0)
          if x > 0 and y > 0:
             v = min(v, D[y - 1, x - 1] + 1.414)
          if x + 1 < w and y > 0:
             v = min(v, D[y - 1, x + 1] + 1.414)
          D[y, x] = v
    for y in range(h - 1, -1, -1):
       for x in range(w - 1, -1, -1):
          v = D[y, x]
          if x + 1 < w:
             v = min(v, D[y, x + 1] + 1.0)
          if y + 1 < h:
             v = min(v, D[y + 1, x] + 1.0)
          if x + 1 < w and y + 1 < h:
             v = min(v, D[y + 1, x + 1] + 1.414)
          if x > 0 and y + 1 < h:
             v = min(v, D[y + 1, x - 1] + 1.414)
          D[y, x] = v
    return D


def detect_oval_bbox(rgb):
    h, w, _ = rgb.shape
    white = (rgb[:, :, 0] > 250) & (rgb[:, :, 1] > 250) & (rgb[:, :, 2] > 250)
    oval_left = np.full(h, np.nan)
    oval_right = np.full(h, np.nan)
    for y in range(h):
       row = ~white[y]
       d = np.diff(np.r_[0, row.astype(int), 0])
       starts = np.where(d == 1)[0]
       ends = np.where(d == -1)[0] - 1
       if len(starts) == 0:
          continue
       lengths = ends - starts + 1
```

```
        candidates = [
            (int(lengths[i]), int(starts[i]), int(ends[i]))
            for i in range(len(starts))
            if starts[i] < 0.72 * w and lengths[i] > 40
        ]
        if not candidates:
            continue
        candidates.sort(reverse=True)
        _, L, R = candidates[0]
        oval_left[y] = L
        oval_right[y] = R
    widths = np.where(np.isfinite(oval_left), oval_right - oval_left, 0.0)
    yeq = int(np.argmax(widths))
    thresh = 0.08 * widths[yeq]
    y0 = yeq
    while y0 > 0 and widths[y0 - 1] >= thresh:
        y0 -= 1
    y1 = yeq
    while y1 < h - 1 and widths[y1 + 1] >= thresh:
        y1 += 1
    x0 = float(oval_left[yeq])
    x1 = float(oval_right[yeq])
    return {
        "x0": int(x0),
        "x1": int(x1),
        "y0": int(y0),
        "y1": int(y1),
        "yeq": int(yeq),
        "cx": 0.5 * (x0 + x1),
        "cy": 0.5 * (y0 + y1),
        "Rx": 0.5 * (x1 - x0),
        "Ry": 0.5 * (y1 - y0 + 1),
    }


def refine_registration(rgb, seed):
    """Gray/dark Dec=0 lock (pre medial-axis). Prefer GOOD_REG hard-set in main."""
    h, w, _ = rgb.shape
    dark = rgb.mean(axis=2) < 55
    dist = chamfer_distance_to_dark(dark)

    ell_eq, b_eq = celestial_equator_lb(2500)
    lon = np.deg2rad(lon_disp_from_ell(ell_eq))
    lat = np.deg2rad(b_eq)
    mx, my = mollweide_xy(lon, lat)
    Xn = mx / (2.0 * np.sqrt(2.0))
    Yn = my / np.sqrt(2.0)

    def median_dist(cx, cy, Rx, Ry):
        px = cx + Xn * Rx
        py = cy - Yn * Ry
        m = (px >= 1) & (px < w - 1) & (py >= 1) & (py < h - 1)
        xi = np.clip(np.round(px[m]).astype(int), 0, w - 1)
        yi = np.clip(np.round(py[m]).astype(int), 0, h - 1)
        return float(np.median(dist[yi, xi]))

    base = {k: float(seed[k]) for k in ("cx", "cy", "Rx", "Ry")}
    best_sc = median_dist(**base)
    best = dict(base)
    for dcx in np.linspace(-10, 10, 21):
        for dcy in np.linspace(-10, 10, 21):
            for sRx in (0.98, 0.99, 1.0, 1.01, 1.02):
                for sRy in (0.98, 0.99, 1.0, 1.01, 1.02):
                    p = dict(
                        cx=base["cx"] + dcx,
                        cy=base["cy"] + dcy,
                        Rx=base["Rx"] * sRx,
                        Ry=base["Ry"] * sRy,
                    )
                    sc = median_dist(**p)
                    if sc < best_sc:
                        best_sc = sc
                        best = p
    b = best
    for dcx in np.linspace(-2, 2, 17):
        for dcy in np.linspace(-2, 2, 17):
            for sRx in np.linspace(0.995, 1.005, 11):
                for sRy in np.linspace(0.995, 1.005, 11):
                    p = dict(
                        cx=b["cx"] + dcx,
                        cy=b["cy"] + dcy,
                        Rx=b["Rx"] * sRx,
                        Ry=b["Ry"] * sRy,
                    )
                    sc = median_dist(**p)
                    if sc < best_sc:
                        best_sc = sc
                        best = p
```

```
    best["ceq_median_px"] = float(best_sc)
    return best


def choose_source():
    for p in SOURCE_CANDIDATES:
        if p.is_file():
            return p
    raise FileNotFoundError("No Migkas Fig.8 theta=75 source image found")


def sample_rgb(rgb, ell, b, cx, cy, Rx, Ry, r=4):
    px, py = lb_to_px(ell, b, cx, cy, Rx, Ry)
    xi, yi = int(round(as_xy(px, py)[0])), int(round(as_xy(px, py)[1]))
    patch = rgb[max(0, yi - r) : yi + r + 1, max(0, xi - r) : xi + r + 1]
    return patch.mean(axis=(0, 1))


def validate_landmarks(rgb, reg):
    cx, cy, Rx, Ry = reg["cx"], reg["cy"], reg["Rx"], reg["Ry"]
    pts = {}
    landmarks = [
        ("center", 0.0, 0.0),
        ("ell90", 90.0, 0.0),
        ("ell270", 270.0, 0.0),
        ("north", 0.0, 90.0),
        ("south", 0.0, -90.0),
    ]
    for i, (ell, b) in enumerate(MIGKAS_MAXES):
        landmarks.append((f"migkas_max{i}", ell, b))
    for i, (ell, b) in enumerate(MIGKAS_MINS):
        landmarks.append((f"migkas_min{i}", ell, b))
    for name, ell, b in landmarks:
        x, y = as_xy(*lb_to_px(ell, b, cx, cy, Rx, Ry))
        pts[name] = {"ell": ell, "b": b, "px": x, "py": y}

    assert pts["ell90"]["px"] < pts["center"]["px"], "l=90 must be LEFT of center"
    assert pts["ell270"]["px"] > pts["center"]["px"], "l=270 must be RIGHT of center"
    assert abs(pts["center"]["px"] - cx) < 1.5
    assert abs(pts["center"]["py"] - cy) < 1.5
    assert pts["north"]["py"] < pts["center"]["py"]
    assert pts["south"]["py"] > pts["center"]["py"]

    # Map-extrema pair: (17,-9) should sit in a warmer patch than (274,-22).
    cmax = sample_rgb(rgb, *MIGKAS_MAXES[0], cx, cy, Rx, Ry)
    cmin = sample_rgb(rgb, *MIGKAS_MINS[1], cx, cy, Rx, Ry)
    assert (cmax[0] + cmax[1]) > (cmin[0] + cmin[1]) + 50
    return {
        "landmarks_px": pts,
        "migkas_max0_rgb_mean": [float(x) for x in cmax],
        "migkas_min1_rgb_mean": [float(x) for x in cmin],
    }


def inside_ellipse(px, py, ecx, ecy, eRx, eRy, r_lim=1.0):
    """Boolean mask: points inside axis-aligned ellipse (image y down)."""
    px = np.asarray(px, dtype=float)
    py = np.asarray(py, dtype=float)
    Xn = (px - ecx) / eRx
    Yn = (py - ecy) / eRy
    return Xn * Xn + Yn * Yn <= float(r_lim) * float(r_lim)


def plot_lb_curve(
    ax, ell, b, cx, cy, Rx, Ry, inset=1.0, jump_px=45.0,
    clip_r2=None, clip_path=None, color_oval=None, **kw
):
    """Draw curve in oval; inset<1 keeps strokes inside the published Mollweide edge."""
    px, py = lb_to_px(ell, b, cx, cy, Rx, Ry)
    px = np.asarray(px, dtype=float).reshape(-1)
    py = np.asarray(py, dtype=float).reshape(-1)
    # Pull points slightly toward oval center so thick strokes do not spill past the rim.
    px = cx + (px - cx) * inset
    py = cy + (py - cy) * inset
    Xn = (px - cx) / Rx
    Yn = (cy - py) / Ry
    r2_lim = (inset * inset) if clip_r2 is None else float(clip_r2)
    inside = Xn * Xn + Yn * Yn <= r2_lim
    # Also require published colored oval (seed) so registration oversize cannot bleed.
    if color_oval is not None:
        ocx, ocy, oRx, oRy, o_r = color_oval
        inside = inside & inside_ellipse(px, py, ocx, ocy, oRx, oRy, o_r)
    px = np.where(inside, px, np.nan)
    py = np.where(inside, py, np.nan)

    segs = []
    cur_x, cur_y = [], []
    for x, y in zip(px, py):
```

```
        if not (np.isfinite(x) and np.isfinite(y)):
            if len(cur_x) >= 2:
                segs.append((cur_x, cur_y))
            cur_x, cur_y = [], []
            continue
        if cur_x and np.hypot(x - cur_x[-1], y - cur_y[-1]) > jump_px:
            if len(cur_x) >= 2:
                segs.append((cur_x, cur_y))
            cur_x, cur_y = [float(x)], [float(y)]
        else:
            cur_x.append(float(x))
            cur_y.append(float(y))
    if len(cur_x) >= 2:
        segs.append((cur_x, cur_y))
    if clip_path is not None:
        kw["clip_path"] = clip_path
        kw["clip_on"] = True
    for sx, sy in segs:
        (line,) = ax.plot(sx, sy, **kw)
        if clip_path is not None:
            line.set_clip_path(clip_path)
            line.set_clip_on(True)


def plot_closed_ring(
    ax, mu, cx, cy, Rx, Ry, color, lw, inset=1.0, jump_px=80.0,
    clip_path=None, color_oval=None, ls="-", wash_alpha=None,
):
    """
    Exact projected small-circle about D_HAT=+x: n̂·x = μ.

    Parameterize on the sphere, convert to Galactic □(,b), apply
    lon_disp = -wrap□(), Mollweide-project, and stroke continuous polylines.
    Split wherever display-longitude jumps across the antimeridian (or pixel
    distance jumps). Never invent interior chords, ellipse fits, or decorative
    oval-rim bridges.

    Notes on θlabels used elsewhere:μ
      =cos(30°) →r=30° about +x (closed oval near map center)μ
      =cos(185°)=cos(175°) →r=175° about +x ≡5° about CoE→
        small arcs on BOTH left and right map edges (not a rim loop)μ
      =cos(115°) →r=115° about +x (large seam-crossing curve)μ
      =cos(265°)=cos(95°) →r=95° about +x (near-meridian, seam-crossing)
    """
    pts = small_circle_about_axis(D_HAT, mu, n=4001)
    # Keep sampling order around the axis (azimuth) —stable for splitting.
    phi = np.arctan2(pts[:, 2], pts[:, 1])
    order = np.argsort(phi)
    pts = pts[order]

    ell = np.empty(len(pts))
    b = np.empty(len(pts))
    for i, p in enumerate(pts):
        ell[i], b[i] = n_to_lb(p)

    lon = np.deg2rad(lon_disp_from_ell(ell)) # λ∈π(-, π]
    lat = np.deg2rad(b)

    # Split on antimeridian jumps in display longitude BEFORE pixel projection.
    dlon = np.diff(lon)
    cuts = np.where(np.abs(dlon) > np.pi)[0] + 1
    # Also close the periodic sample: check wrap between last and first.
    # Build open segments from azimuth-ordered points.
    idx = np.concatenate([[0], cuts, [len(lon)]])
    lon_segs = []
    for i0, i1 in zip(idx[:-1], idx[1:]):
        if i1 - i0 >= 2:
            lon_segs.append((lon[i0:i1].copy(), lat[i0:i1].copy()))

    # Circle sampling is cyclic. Merge the artificial →lastfirst break when
    # that periodic join is not itself an antimeridian crossing. Otherwise a
    # seam-crossing circle falsely appears as three pieces with a tiny hole.
    if len(lon_segs) > 1 and abs(lon[0] - lon[-1]) <= np.pi:
        first_lo, first_la = lon_segs[0]
        last_lo, last_la = lon_segs[-1]
        lon_segs = [(
            np.concatenate([last_lo, first_lo]),
            np.concatenate([last_la, first_la]),
        )] + lon_segs[1:-1]

    # Only now pin TRUE seam endpoints to π±. Never snap the arbitrary
    # parameterization start/end to the boundary.
    snapped = []
    for lo, la in lon_segs:
        if abs(abs(lo[0]) - np.pi) < np.deg2rad(10):
            lo[0] = np.sign(lo[0] if lo[0] != 0 else 1.0) * np.pi
        if abs(abs(lo[-1]) - np.pi) < np.deg2rad(10):
            lo[-1] = np.sign(lo[-1] if lo[-1] != 0 else 1.0) * np.pi
```

```
        snapped.append((lo, la))
    lon_segs = snapped

    def stroke_xy(xs, ys):
        if len(xs) < 2:
            return
        if wash_alpha is not None:
            # Soft μ-wash stroke: no white underlay; translucent color only.
            layers = ((color, lw, 2, float(wash_alpha)),)
        else:
            # Clean mid-stroke: optional thin white underlay only (lw+1.5), then color.
            # No black outer rim, no thick white halo stack.
            layers = (
                ("white", lw + 1.5, 3, 0.95),
                (color, lw, 4, 1.0),
            )
        for col, width, z, a in layers:
            (line,) = ax.plot(
                xs, ys, color=col, lw=width, linestyle=ls,
                solid_capstyle="round", solid_joinstyle="round",
                zorder=z, alpha=a,
                clip_path=clip_path, clip_on=True,
            )
            if clip_path is not None:
                line.set_clip_path(clip_path)
                line.set_clip_on(True)

    r2_lim = inset * inset + 1e-10
    drawn = 0
    for lo, la in lon_segs:
        # Mollweide in normalized oval coords, then pixels.
        x_m, y_m = mollweide_xy(lo, la)
        Xn = x_m / (2.0 * np.sqrt(2.0))
        Yn = y_m / np.sqrt(2.0)
        px = cx + Xn * Rx
        py = cy - Yn * Ry
        # Mild inset so thick strokes stay inside published oval (no rim-snap).
        px = cx + (px - cx) * inset
        py = cy + (py - cy) * inset
        Xn2 = (px - cx) / Rx
        Yn2 = (cy - py) / Ry
        inside = (Xn2 * Xn2 + Yn2 * Yn2) <= r2_lim
        if color_oval is not None:
            ocx, ocy, oRx, oRy, o_r = color_oval
            inside = inside & inside_ellipse(px, py, ocx, ocy, oRx, oRy, o_r)

        # Pixel-space split on NaN / large jumps / outside oval.
        cur_x, cur_y = [], []
        for x, y, ok in zip(px, py, inside):
            if (not ok) or (not np.isfinite(x)) or (not np.isfinite(y)):
                if len(cur_x) >= 2:
                    stroke_xy(cur_x, cur_y)
                    drawn += 1
                cur_x, cur_y = [], []
                continue
            if cur_x and np.hypot(x - cur_x[-1], y - cur_y[-1]) > jump_px:
                if len(cur_x) >= 2:
                    stroke_xy(cur_x, cur_y)
                    drawn += 1
                cur_x, cur_y = [float(x)], [float(y)]
            else:
                cur_x.append(float(x))
                cur_y.append(float(y))
        if len(cur_x) >= 2:
            # Close only if this single segment is already nearly a loop
            # (non-wrapping ring such as MAX 30°).
            if (
                drawn == 0
                and len(lon_segs) == 1
                and np.hypot(cur_x[0] - cur_x[-1], cur_y[0] - cur_y[-1]) < jump_px
            ):
                cur_x.append(cur_x[0])
                cur_y.append(cur_y[0])
            stroke_xy(cur_x, cur_y)
            drawn += 1


def _ring_px_arrays(mu, cx, cy, Rx, Ry, inset, n_pts=721):
    """Small-circle n·Dμ= →pixel (px, py) arrays, azimuth-ordered, with inset."""
    pts = small_circle_about_axis(D_HAT, float(mu), n=n_pts)
    phi = np.arctan2(pts[:, 2], pts[:, 1])
    order = np.argsort(phi)
    pts = pts[order]
    ell = np.empty(n_pts)
    b = np.empty(n_pts)
    for i, p in enumerate(pts):
        ell[i], b[i] = n_to_lb(p)
```

```
    lon = np.deg2rad(lon_disp_from_ell(ell))
    lat = np.deg2rad(b)
    x_m, y_m = mollweide_xy(lon, lat)
    Xn = x_m / (2.0 * np.sqrt(2.0))
    Yn = y_m / np.sqrt(2.0)
    px = cx + Xn * Rx
    py = cy - Yn * Ry
    px = cx + (px - cx) * inset
    py = cy + (py - cy) * inset
    return lon, px, py


def fill_highlight_band(
    ax, theta_lo, theta_hi, theta_mid, cx, cy, Rx, Ry, color,
    inset=1.0, clip_path=None, color_oval=None, n_pts=721, jump_px=80.0,
):
    """
    One highlight band matching three-panel Panel B:
      continuous transparent polygon strips between ±BAND_HALF edges
      (broken only at Mollweide antimeridian / large pixel jumps —NOT a dense
      multi-stroke wash) + ONE solid mid-stroke at θ_mid.
    """
    mu_a, mu_b = mu_pair_for_band(theta_lo, theta_hi)
    mu_mid = float(np.cos(np.deg2rad(theta_mid)))
    lon_a, px_a, py_a = _ring_px_arrays(mu_a, cx, cy, Rx, Ry, inset, n_pts=n_pts)
    lon_b, px_b, py_b = _ring_px_arrays(mu_b, cx, cy, Rx, Ry, inset, n_pts=n_pts)

    r2_lim = inset * inset + 1e-10

    def _inside(px, py):
        Xn2 = (px - cx) / Rx
        Yn2 = (cy - py) / Ry
        ok = (Xn2 * Xn2 + Yn2 * Yn2) <= r2_lim
        if color_oval is not None:
            ocx, ocy, oRx, oRy, o_r = color_oval
            ok = ok & inside_ellipse(px, py, ocx, ocy, oRx, oRy, o_r)
        return ok

    gap = 0.9 * np.pi
    bad = np.zeros(n_pts - 1, dtype=bool)
    for i in range(n_pts - 1):
        if abs(lon_a[i + 1] - lon_a[i]) > gap:
            bad[i] = True
        elif abs(lon_b[i + 1] - lon_b[i]) > gap:
            bad[i] = True
        elif abs(lon_a[i] - lon_b[i]) > gap:
            bad[i] = True
        elif abs(lon_a[i + 1] - lon_b[i + 1]) > gap:
            bad[i] = True
        elif np.hypot(px_a[i + 1] - px_a[i], py_a[i + 1] - py_a[i]) > jump_px:
            bad[i] = True
        elif np.hypot(px_b[i + 1] - px_b[i], py_b[i + 1] - py_b[i]) > jump_px:
            bad[i] = True
        elif (not _inside(px_a[i], py_a[i])) or (not _inside(px_b[i], py_b[i])):
            bad[i] = True
        elif (not _inside(px_a[i + 1], py_a[i + 1])) or (not _inside(px_b[i + 1], py_b[i + 1])):
            bad[i] = True

    polys = []
    i = 0
    while i < n_pts - 1:
        while i < n_pts - 1 and bad[i]:
            i += 1
        if i >= n_pts - 1:
            break
        j = i
        while j < n_pts - 1 and not bad[j]:
            j += 1
        i0, i1 = i, j
        if i1 - i0 >= 1:
            xs = np.concatenate([px_a[i0:i1 + 1], px_b[i0:i1 + 1][::-1]])
            ys = np.concatenate([py_a[i0:i1 + 1], py_b[i0:i1 + 1][::-1]])
            polys.append(np.column_stack([xs, ys]))
        i = j + 1

    if polys:
        # Soft transparent highlight (clean pre-halo look).
        coll = PolyCollection(
            polys, facecolors=color, edgecolors="none",
            alpha=0.30, zorder=2, linewidths=0,
        )
        ax.add_collection(coll)
        if clip_path is not None:
            coll.set_clip_path(clip_path)
            coll.set_clip_on(True)

    # ONE solid mid-stroke only (thin white underlay lw+1.5). No boundary-edge
    # strokes —dual edges with halos multiply visual lines into "extra loops".
```

```
    plot_closed_ring(
        ax, mu_mid, cx, cy, Rx, Ry, color, lw=2.8,
        inset=inset, clip_path=clip_path, color_oval=color_oval,
    )



# --- Opacity gradient (same rule as make_coe_rings_migkas_pin_axis_matched_grad*.py):
# alpha *= (1+cos psi)/2, psi = ring azimuth from the θ-ray point (ring point most
# along sign(sin θ)Ŷ·, Ŷ= Galactic +z projected ⊥D), rolled by ROLL (Y' = cY −sZ).
from matplotlib.collections import LineCollection
from matplotlib.colors import to_rgba


def _ring_basis_roll(D, roll_deg):
    a = D / np.linalg.norm(D)
    tmp = np.array([0.0, 0.0, 1.0])
    if abs(np.dot(a, tmp)) > 0.9:
        tmp = np.array([0.0, 1.0, 0.0])
    y = tmp - np.dot(tmp, a) * a
    y /= np.linalg.norm(y)
    z = np.cross(a, y)
    z /= np.linalg.norm(z)
    r = np.deg2rad(float(roll_deg))
    c, s_ = np.cos(r), np.sin(r)
    return c * y - s_ * z, s_ * y + c * z


def _grad_ring_px(mu, theta_mid, cx, cy, Rx, Ry, inset, n):
    a = D_HAT / np.linalg.norm(D_HAT)
    mu = float(np.clip(mu, -1.0 + 1e-9, 1.0 - 1e-9))
    rho = np.sqrt(1.0 - mu * mu)
    y, z = _ring_basis_roll(a, ROLL)
    phi0 = 0.5 * np.pi if np.sin(np.deg2rad(theta_mid)) >= 0 else 1.5 * np.pi
    phi = phi0 + np.linspace(-np.pi, np.pi, n)
    pts = (mu * a)[None, :] + rho * (np.outer(np.sin(phi), y) + np.outer(np.cos(phi), z))
    ell = np.array([n_to_lb(p)[0] for p in pts]); b = np.array([n_to_lb(p)[1] for p in pts])
    lon = np.deg2rad(lon_disp_from_ell(ell)); lat = np.deg2rad(b)
    x_m, y_m = mollweide_xy(lon, lat)
    px = cx + x_m / (2.0 * np.sqrt(2.0)) * Rx
    py = cy - y_m / np.sqrt(2.0) * Ry
    px = cx + (px - cx) * inset; py = cy + (py - cy) * inset
    return lon, px, py, 0.5 * (1.0 + np.cos(phi - phi0))


def _rgba(color, alphas):
    r, g, b_, _ = to_rgba(color)
    al = np.clip(np.asarray(alphas, dtype=float), 0.0, 1.0)
    out = np.empty((al.size, 4)); out[:, 0], out[:, 1], out[:, 2], out[:, 3] = r, g, b_, al
    return out


def _runs(seg_sc, valid, n_levels=64):
    lev = np.round(np.clip(seg_sc, 0.0, 1.0) * n_levels) / n_levels
    ok = (lev > 0) & valid
    runs, i, n = [], 0, lev.size
    while i < n:
        if not ok[i]:
            i += 1; continue
        j = i + 1
        while j < n and ok[j] and lev[j] == lev[i]:
            j += 1
        runs.append((i, j, float(lev[i]))); i = j
    return runs


def fill_highlight_band_grad(
    ax, theta_lo, theta_hi, theta_mid, cx, cy, Rx, Ry, color,
    inset=1.0, clip_path=None, color_oval=None, n_pts=721, jump_px=80.0,
):
    """Same band (alpha 0.30) + mid-stroke (white lw+1.5 @0.95 under colour lw 2.8)
    as fill_highlight_band, but with the (1+cos psi)/2 opacity gradient."""
    mu_a, mu_b = mu_pair_for_band(theta_lo, theta_hi)
    mu_mid = float(np.cos(np.deg2rad(theta_mid)))
    r2_lim = inset * inset + 1e-10

    def _inside(px, py):
        ok = ((px - cx) / Rx) ** 2 + ((cy - py) / Ry) ** 2 <= r2_lim
        if color_oval is not None:
            ocx, ocy, oRx, oRy, o_r = color_oval
            ok = ok & inside_ellipse(px, py, ocx, ocy, oRx, oRy, o_r)
        return ok

    def _clip(coll):
        if clip_path is not None:
            coll.set_clip_path(clip_path); coll.set_clip_on(True)
        ax.add_collection(coll)

```

```
    gap = 0.9 * np.pi
    lon_a, pxa, pya, sc = _grad_ring_px(mu_a, theta_mid, cx, cy, Rx, Ry, inset, n_pts)
    lon_b, pxb, pyb, _ = _grad_ring_px(mu_b, theta_mid, cx, cy, Rx, Ry, inset, n_pts)
    qa = np.stack([lon_a[:-1], lon_a[1:], lon_b[1:], lon_b[:-1]])
    ins_a, ins_b = _inside(pxa, pya), _inside(pxb, pyb)
    valid = ((qa.max(0) - qa.min(0)) <= gap)
    valid &= np.hypot(np.diff(pxa), np.diff(pya)) <= jump_px
    valid &= np.hypot(np.diff(pxb), np.diff(pyb)) <= jump_px
    valid &= ins_a[:-1] & ins_a[1:] & ins_b[:-1] & ins_b[1:]
    runs = _runs(0.5 * (sc[:-1] + sc[1:]), valid)
    if runs:
       polys = [np.column_stack([np.concatenate([pxa[i0:i1 + 1], pxb[i0:i1 + 1][::-1]]),
                         np.concatenate([pya[i0:i1 + 1], pyb[i0:i1 + 1][::-1]])])
             for i0, i1, _ in runs]
       _clip(PolyCollection(polys, facecolors=_rgba(color, [0.30 * r[2] for r in runs]),
                       edgecolors="none", linewidths=0, zorder=2))
    lon_m, pxm, pym, scm = _grad_ring_px(mu_mid, theta_mid, cx, cy, Rx, Ry, inset, 4001)
    ins = _inside(pxm, pym)
    valid = (np.abs(np.diff(lon_m)) <= np.pi) & (np.hypot(np.diff(pxm), np.diff(pym)) <= jump_px)
    valid &= ins[:-1] & ins[1:]
    runs = _runs(0.5 * (scm[:-1] + scm[1:]), valid)
    if runs:
       segs = [np.column_stack([pxm[i0:i1 + 1], pym[i0:i1 + 1]]) for i0, i1, _ in runs]
       _clip(LineCollection(segs, colors=_rgba("white", [0.95 * r[2] for r in runs]),
                       linewidths=2.8 + 1.5, zorder=3, capstyle="butt", joinstyle="round"))
       _clip(LineCollection(segs, colors=_rgba(color, [1.0 * r[2] for r in runs]),
                       linewidths=2.8, zorder=4, capstyle="butt", joinstyle="round"))


def draw_overlay(rgb, reg, seed, source):

    h, w, _ = rgb.shape
    cx, cy, Rx, Ry = reg["cx"], reg["cy"], reg["Rx"], reg["Ry"]
    # Published colored Mollweide (seed detect) —used to keep overlays inside
    # the Migkas map even when registration ellipse is slightly larger.
    scx = float(seed["cx"])
    scy = float(seed["cy"])
    sRx = float(seed["Rx"])
    sRy = float(seed["Ry"])

    dpi = 120
    _extra_h = 1.35 if SHOW_LEGEND else 0.35 # tighten when legend omitted
    fig, ax = plt.subplots(figsize=(w / dpi, (h / dpi) + _extra_h), dpi=dpi)
    ax.imshow(rgb, origin="upper", extent=(0, w, h, 0), interpolation="nearest")
    ax.set_xlim(0, w)
    ax.set_ylim(h, 0)
    ax.set_aspect("equal")
    ax.axis("off")

    # clip_path = registered oval (cx,cy,Rx,Ry). Registration itself is unchanged.
    # Because the refined ellipse can sit a bit outside the published colored
    # map, we ALSO inset strokes and cull/inset markers against the seed oval.
    clip_oval = Ellipse(
       (cx, cy), width=2.0 * Rx, height=2.0 * Ry,
       facecolor="none", edgecolor="none", transform=ax.transData,
    )
    ax.add_patch(clip_oval)

    # Stroke inset so thick white+color underlays do not AA-bleed past the rim.
    STROKE_INSET = 0.97
    # Cull points outside this fraction of the published colored seed oval.
    COLOR_R = 0.978
    color_oval = (scx, scy, sRx, sRy, COLOR_R)

    def stroke_curve(ell, b, color, lw, ls="-"):
       # White underlay then color —stands out on magma without looking oversized.
       plot_lb_curve(
          ax, ell, b, cx, cy, Rx, Ry, inset=STROKE_INSET, clip_path=clip_oval,
          color_oval=color_oval,
          color="white", lw=lw + 1.6, ls=ls, alpha=0.95,
          solid_capstyle="round", zorder=3,
       )
       plot_lb_curve(
          ax, ell, b, cx, cy, Rx, Ry, inset=STROKE_INSET, clip_path=clip_oval,
          color_oval=color_oval,
          color=color, lw=lw, ls=ls, alpha=1.0,
          solid_capstyle="round", zorder=4,
       )

    def mark_point(ell, b, marker, color, ms):
       """Place marker; pull limb symbols inward so halo stays in colored oval."""
       px, py = as_xy(*lb_to_px(ell, b, cx, cy, Rx, Ry))
       # Approx marker+halo radius in pixels (ms is points).
       halo_ms = ms + 1.8
       ms_px = 0.5 * halo_ms * (dpi / 72.0)
       # Normalized margin inside the published colored oval.
       margin = ms_px / min(sRx, sRy) + 0.012
       r_lim = max(0.55, COLOR_R - margin)
```

```
        Xn = (px - scx) / sRx
        Yn = (py - scy) / sRy
        r_col = float(np.hypot(Xn, Yn))

        if r_col > 1e-9:
           # Scale toward colored-oval center so the whole symbol fits.
           if r_col > r_lim:
              scale = r_lim / r_col
              px = scx + (px - scx) * scale
              py = scy + (py - scy) * scale
              Xn = (px - scx) / sRx
              Yn = (py - scy) / sRy
              r_col = float(np.hypot(Xn, Yn))
           # Mild extra inset for everyone (matches stroke inset spirit).
           mild = min(STROKE_INSET, 0.995)
           px = scx + (px - scx) * mild
           py = scy + (py - scy) * mild
           Xn = (px - scx) / sRx
           Yn = (py - scy) / sRy
           r_col = float(np.hypot(Xn, Yn))

        # Skip if still too close to the rim for this marker size.
        if r_col + margin > COLOR_R + 1e-6:
           return

        def _mark(m_size, m_color, m_edge, m_ew, z):
           (line,) = ax.plot(
              px, py, marker=marker, ms=m_size, color=m_color,
              markeredgecolor=m_edge, markeredgewidth=m_ew, linestyle="None",
              zorder=z, clip_path=clip_oval, clip_on=True,
           )
           line.set_clip_path(clip_oval)
           line.set_clip_on(True)

        _mark(ms + 1.8, "white", "white", 0, 6)
        _mark(ms, color, "k", 0.55, 7)

     # CoE axis at geometric inset=1.0. Do NOT overlay Dec=0 —Migkas already
     # printed the celestial equator on Fig. 8; a second green dashed stroke fights it.
     def stroke_curve_geom(ell, b, color, lw, ls="-"):
        plot_lb_curve(
           ax, ell, b, cx, cy, Rx, Ry, inset=1.0, clip_path=clip_oval,
           color_oval=color_oval,
           color="white", lw=lw + 1.6, ls=ls, alpha=0.95,
           solid_capstyle="round", zorder=3,
        )
        plot_lb_curve(
           ax, ell, b, cx, cy, Rx, Ry, inset=1.0, clip_path=clip_oval,
           color_oval=color_oval,
           color=color, lw=lw, ls=ls, alpha=1.0,
           solid_capstyle="round", zorder=4,
        )

     # ↔CoEanti-CoE axis = great circle through ±D (tilted +30°; not Galactic equator)
     _ax_ell, _ax_b = great_circle_through_poles(D_HAT, n=721)
     stroke_curve_geom(_ax_ell, _ax_b, C_AXIS, lw=1.15)

     # Exactly four highlight bands (match three-panel Panel B): each ±9°
     # (= 18° end-to-end = 5% of 360°) + one solid mid-stroke. No companion.
     for (lo, hi), mid, color in zip(MAX_BANDS, THETA_MAX, COLORS_MAX):
        (fill_highlight_band_grad if GRAD else fill_highlight_band)(
           ax, lo, hi, mid, cx, cy, Rx, Ry, color,
           inset=STROKE_INSET, clip_path=clip_oval, color_oval=color_oval,
        )
     for (lo, hi), mid, color in zip(MIN_BANDS, THETA_MIN, COLORS_MIN):
        (fill_highlight_band_grad if GRAD else fill_highlight_band)(
           ax, lo, hi, mid, cx, cy, Rx, Ry, color,
           inset=STROKE_INSET, clip_path=clip_oval, color_oval=color_oval,
        )

     # Markers (limb ones get stronger inset inside mark_point); per-loop colors
     mark_point(ANTI_COE_LB[0], ANTI_COE_LB[1], "o", C_ANTI, ms=5.5)
     mark_point(COE_LB[0], COE_LB[1], "*", C_COE, ms=7.5)
     for (ell, b), color in zip(MIGKAS_MAXES, COLORS_MAX):
        mark_point(ell, b, "^", color, ms=6.5)
     for (ell, b), color in zip(MIGKAS_MINS, COLORS_MIN):
        mark_point(ell, b, "s", color, ms=6.0)

     handles = [
        Patch(
           facecolor=C_MAX_30, alpha=0.55, edgecolor=C_MAX_30,
           label=(
              rf"MAX $\theta\!=\!{THETA_MAX[0]:g}^\circ$ band "
              rf"[{MAX_BANDS[0][0]:g},{MAX_BANDS[0][1]:g}]$^\circ$ "
              rf"($18^\circ$=$\pm9^\circ$=5% of $360^\circ$)"
           ),
        ),
```

```
        Patch(
           facecolor=C_MAX_185, alpha=0.55, edgecolor=C_MAX_185,
           label=(
              rf"MAX $\theta\!=\!{THETA_MAX[1]:g}^\circ$ band "
              rf"[{MAX_BANDS[1][0]:g},{MAX_BANDS[1][1]:g}]$^\circ$ "
              rf"($18^\circ$=$\pm9^\circ$)"
           ),
        ),
        Patch(
           facecolor=C_MIN_115, alpha=0.55, edgecolor=C_MIN_115,
           label=(
              rf"MIN $\theta\!=\!{THETA_MIN[0]:g}^\circ$ band "
              rf"[{MIN_BANDS[0][0]:g},{MIN_BANDS[0][1]:g}]$^\circ$ "
              rf"($18^\circ$=$\pm9^\circ$)"
           ),
        ),
        Patch(
           facecolor=C_MIN_265, alpha=0.55, edgecolor=C_MIN_265,
           label=(
              rf"MIN $\theta\!=\!{THETA_MIN[1]:g}^\circ$ band "
              rf"[{MIN_BANDS[1][0]:g},{MIN_BANDS[1][1]:g}]$^\circ$ "
              rf"($18^\circ$=$\pm9^\circ$)"
           ),
        ),
        Line2D([0], [0], color=C_AXIS, lw=2.0, label=r"CoE axis tilted $+15^\circ$"),
        Line2D(
           [0], [0], marker="^", color="w", markerfacecolor=C_MAX_30,
           markeredgecolor="k", linestyle="None", ms=8,
           label=rf"MAX $({MIGKAS_MAXES[0][0]:.0f}^\circ,{MIGKAS_MAXES[0][1]:+.0f}^\circ)$",
        ),
        Line2D(
           [0], [0], marker="^", color="w", markerfacecolor=C_MAX_185,
           markeredgecolor="k", linestyle="None", ms=8,
           label=rf"MAX $({MIGKAS_MAXES[1][0]:.0f}^\circ,{MIGKAS_MAXES[1][1]:+.0f}^\circ)$",
        ),
        Line2D(
           [0], [0], marker="s", color="w", markerfacecolor=C_MIN_115,
           markeredgecolor="k", linestyle="None", ms=7,
           label=rf"MIN $({MIGKAS_MINS[0][0]:.0f}^\circ,{MIGKAS_MINS[0][1]:+.0f}^\circ)$",
        ),
        Line2D(
           [0], [0], marker="s", color="w", markerfacecolor=C_MIN_265,
           markeredgecolor="k", linestyle="None", ms=7,
           label=rf"MIN $({MIGKAS_MINS[1][0]:.0f}^\circ,{MIGKAS_MINS[1][1]:+.0f}^\circ)$",
        ),
        Line2D([0], [0], marker="o", color=C_ANTI, markeredgecolor="k",
             linestyle="None", ms=6,
             label=r"anti-CoE $+x$ $(0^\circ,+15^\circ)$"),
        Line2D([0], [0], marker="*", color=C_COE, markeredgecolor="k",
             linestyle="None", ms=9,
             label=r"CoE $(180^\circ,-15^\circ)$"),
    ]
    if SHOW_LEGEND:
        leg = ax.legend(
           handles=handles, loc="upper center", bbox_to_anchor=(0.5, -0.012),
           ncol=4, fontsize=7.5, frameon=True, fancybox=False,
           edgecolor="#222222", framealpha=1.0, facecolor="white",
           labelcolor="#111111", borderpad=0.6, handlelength=2.0,
        )
        leg.get_frame().set_linewidth(1.2)
    fig.suptitle(
        r"Migkas Fig. 8 $\theta=75^\circ$ $A/A_{\rm all}$"
        r" + four CoE highlight bands, each $18^\circ$ end-to-end ($\pm9^\circ$ = 5% of $360^\circ$)"
        "\n"
        r"(two MAX + two MIN; cyan/green/magenta/blue; anti-CoE at $(0^\circ,+15^\circ)$; "
        r"CoE axis tilted $+15^\circ$; $\ell$ increases LEFT"
        + (r"; gradient: darkest at $\theta$ ray, transparent opposite" if GRAD else "")
        + (rf"; ${ROLL:+g}^\circ$ CCW about CoE axis" if abs(ROLL) > 1e-9 else "")
        + ")",
        fontsize=10.5, y=0.985,
    )
    fig.subplots_adjust(left=0.01, right=0.99, top=0.92, bottom=(0.13 if SHOW_LEGEND else 0.02))
    fig.savefig(PNG_PATH, dpi=dpi, facecolor="white")
    plt.close(fig)
    print(f"wrote {PNG_PATH}")


def main():
    source = choose_source()
    print(f"source: {source}")
    rgb = np.asarray(Image.open(source).convert("RGB"))
    seed = detect_oval_bbox(rgb)
    print("oval seed:", {k: seed[k] for k in ("cx", "cy", "Rx", "Ry", "x0", "x1", "y0", "y1")})
    # Restore previous visually-good oval (Chris preferred). No medial-axis Rx stretch.
    reg = dict(GOOD_REG)
    h, w, _ = rgb.shape
    dark = rgb.mean(axis=2) < 55
    dist = chamfer_distance_to_dark(dark)
```

```
    ell_eq, b_eq = celestial_equator_lb(2500)
    lon = np.deg2rad(lon_disp_from_ell(ell_eq))
    lat = np.deg2rad(b_eq)
    mx, my = mollweide_xy(lon, lat)
    Xn = mx / (2.0 * np.sqrt(2.0))
    Yn = my / np.sqrt(2.0)
    px = reg["cx"] + Xn * reg["Rx"]
    py = reg["cy"] - Yn * reg["Ry"]
    m = (px >= 1) & (px < w - 1) & (py >= 1) & (py < h - 1)
    xi = np.clip(np.round(px[m]).astype(int), 0, w - 1)
    yi = np.clip(np.round(py[m]).astype(int), 0, h - 1)
    reg["ceq_median_px"] = float(np.median(dist[yi, xi]))
    reg["registration_source"] = "hard-set GOOD_REG (pre medial-axis retune)"
    print("restored reg:", {k: reg[k] for k in ("cx", "cy", "Rx", "Ry", "ceq_median_px")})
    checks = validate_landmarks(rgb, reg)
    print(
       "validation OK —max0 RGB", checks["migkas_max0_rgb_mean"],
       "min1 RGB", checks["migkas_min1_rgb_mean"],
    )
    global PNG_PATH, JSON_PATH, GRAD, ROLL
    for name, g, r in VERSIONS:
       PNG_PATH = SM_DIR / f"fig8_overlay_{name}.png"
       JSON_PATH = SM_DIR / f"fig8_overlay_{name}.json"
       GRAD, ROLL = g, r
       draw_overlay(rgb, reg, seed, source)
       img = Image.open(PNG_PATH).convert("RGB")
       img.resize((1700, round(img.height * 1700 / img.width)), Image.LANCZOS).save(
          SM_DIR / f"fig8_overlay_{name}_web.jpg", quality=88)

    meta = {
       "png": str(PNG_PATH),
       "script": str(Path(__file__).resolve()),
       "source_image": str(source),
       "convention": (
          "Galactic Mollweide; lon_disp=-wrap(ell); ell increases LEFT "
          "(Migkas Fig.8)"
       ),
       "axis_tilt_b_deg": AXIS_TILT_B_DEG,
       "anti_CoE_lb_deg": [float(ANTI_COE_LB[0]), float(ANTI_COE_LB[1])],
       "CoE_lb_deg": [float(COE_LB[0]), float(COE_LB[1])],
       "D_hat_galactic_xyz": D_HAT.tolist(),
       "axis_tilt_reason": (
          "Chris 2026-09-22 —markers fixed, rings rotate up 30°"
       ),
       "migkas_max_lb_deg": [list(p) for p in MIGKAS_MAXES],
       "migkas_min_lb_deg": [list(p) for p in MIGKAS_MINS],
       "theta_max_from_plus_x_deg": list(THETA_MAX),
       "theta_min_from_plus_x_deg": list(THETA_MIN),
       "band_half_width_deg": BAND_HALF,
       "band_full_width_deg": BAND_FULL,
       "band_width_note": "5% of 360° = 18° end-to-end = ±9° about each MAX/MIN center",
       "max_bands_deg": [list(b) for b in MAX_BANDS],
       "min_bands_deg": [list(b) for b in MIN_BANDS],
       "band_intervals": {
          "MAX_30": [float(MAX_BANDS[0][0]), float(MAX_BANDS[0][1])],
          "MAX_185": [float(MAX_BANDS[1][0]), float(MAX_BANDS[1][1])],
          "MIN_115": [float(MIN_BANDS[0][0]), float(MIN_BANDS[0][1])],
          "MIN_265": [float(MIN_BANDS[1][0]), float(MIN_BANDS[1][1])],
       },
       "colors": {
          "MAX_30": C_MAX_30,
          "MAX_185": C_MAX_185,
          "MIN_115": C_MIN_115,
          "MIN_265": C_MIN_265,
          "palette": "magma-popping cyan/magenta/lime/sky (no reds/oranges)",
       },
       "mu_max_bands": [list(mu_pair_for_band(lo, hi)) for lo, hi in MAX_BANDS],
       "mu_min_bands": [list(mu_pair_for_band(lo, hi)) for lo, hi in MIN_BANDS],
       "scan_stop_source": (
          "best_fit_2ell_multiplier.npz A_after extrema (two max, two min); "
          f"each drawn as one transparent {BAND_FULL:.0f}° end-to-end highlight "
          f"(±{BAND_HALF:.0f}° = 5% of 360°) + one solid mid-stroke; "
          "matches three-panel Panel B; no companion"
       ),
       "matches_three_panel": str(OUT_DIR / "make_coe_rings_migkas_pin_axis.py"),
       "oval_seed_bbox": {
          "x0": seed["x0"], "x1": seed["x1"], "y0": seed["y0"], "y1": seed["y1"],
          "cx": seed["cx"], "cy": seed["cy"], "Rx": seed["Rx"], "Ry": seed["Ry"],
       },
       "registration": {
          "cx": reg["cx"], "cy": reg["cy"], "Rx": reg["Rx"], "Ry": reg["Ry"],
          "ceq_median_px": reg["ceq_median_px"],
          "registration_source": reg.get("registration_source"),
          "ceq_residual_before_registration": dict(GOOD_REG),
          "rejected_medial_axis_reg": {
             "cx": 865.6, "cy": 766.4, "Rx": 706.19, "Ry": 393.70,
             "note": "medial-axis / achromatic DT-ridge retune; blew up Rx; reverted",
          },
```

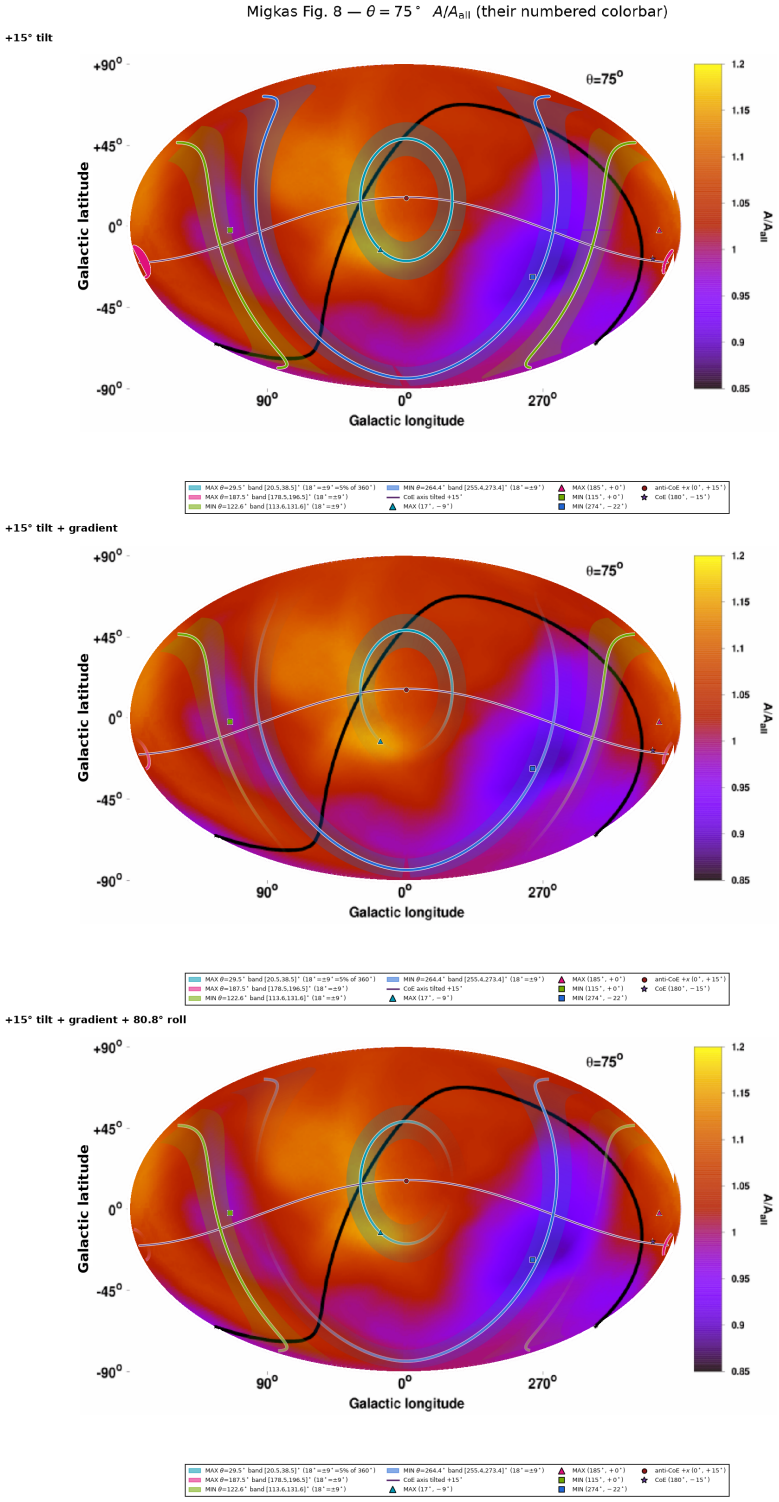
```
            "dofs": ["cx", "cy", "Rx", "Ry"],
            "drawing": {
               "dec0_drawn": False,
               "dec0_note": "not drawn; use Migkas published celestial equator",
               "coe_axis_inset": 1.0,
               "rings_inset": 0.97,
               "note": "CoE axis geometric inset=1; rings mild inset; no Dec=0 overlay",
            },
            "method": (
               "oval seed detect + hard-set to pre-retune visually-good (cx,cy,Rx,Ry); "
               "medial-axis Rx stretch rejected; Dec=0 NOT overlaid (published CEQ kept); "
               "CoE-axis inset=1.0; rings inset~0.97"
            ),
         },
         "landmark_checks": checks,
         "drawing": (
            "Exactly 4 features. Each = ONE smooth transparent PolyCollection "
            "band θ(±9°, 18° end-to-end) + ONE solid mid-stroke at center θ. "
            "Fill ≈alpha0.30. Mid-stroke: thin white underlay (lw+1.5) then color "
            "(lw=2.8). NO black outer rim; NO thick white halo stacks; NO dual "
            "boundary-edge strokes. Colors: cyan/magenta/lime/sky for magma. "
            "No 283° companion. Dec=0 NOT overlaid."
         ),
         "legend_summary": (
            "four highlight bands, each 18° end-to-end (±9° = 5% of 360%); "
            "MAX cyan+magenta, MIN lime+sky; intervals [21,39]/[176,194]/[106,124]/[256,274]"
         ),
         "notes": [
            "Exactly 4 features matching three-panel Panel B geometry/width",
            "Each = ONE smooth transparent PolyCollection ribbon (±9°) + ONE solid mid-stroke only",
            "Clean draw: fill alpha=0.30; mid-stroke thin white underlay (lw+1.5) + color lw=2.8; no black rim; no boundary-edge halos",
            "No multi-stroke scribble wash; no 283° companion; no Dec=0 overlay",
            "MAX bands [21,39] & [176,194]; MIN bands [106,124] & [256,274]",
            "Colors: MAX_30=#00e5ff, MAX_185=#ff2d95, MIN_115=#b8f200, MIN_265=#4fc3f7",
            "band_half_width_deg=9, band_full_width_deg=18 (5% of 360°)",
            "Exact n̂·xμ= small-circles; seam-split polygon fill; mid via plot_closed_ring",
            "No ellipse-fit, no interior chords, no decorative oval-rim loops",
            "No blocking text box on the map; legend below",
            "Colorbar preserved from source (overlays stay on oval)",
            "Overlays clip_path = registered Ellipse(cx,cy,Rx,Ry); rings stroke inset 0.97",
            "Dec=0 NOT drawn (Migkas Fig.8 already has celestial equator); CoE axis = great circle through ±D (tilted +30°) at inset=1.0",
            "axis_tilt_b_deg=30; anti-CoE (0°,+30°), CoE (180°−,30°); Migkas markers fixed at true sky positions",
            "Registration hard-set to pre medial-axis params (863,763.25,693.9927,392.542425); DT-ridge fit rejected",
            "Limb markers inset toward colored-oval center so halo stays inside",
            "Points culled at COLOR_R=0.978 of seed oval; limb markers inset to fit halo",
         ],
      }
      pass # per-version JSON not written (PNG only)
      print(f"wrote {JSON_PATH}")


if __name__ == "__main__":
   main()
```

Figure 5: `fig8_orientation_seq.png` — `split_fig8.py`

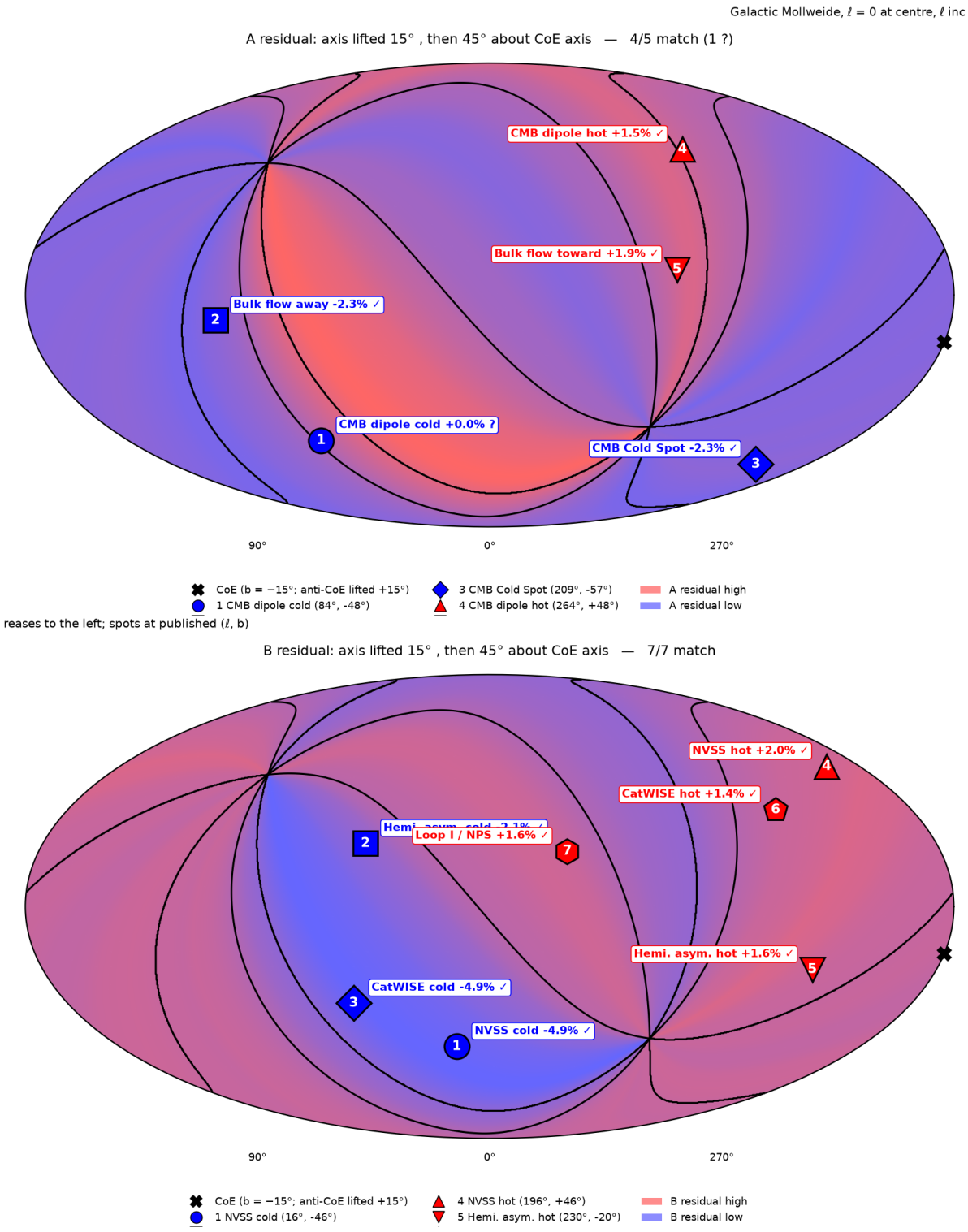
```
# Splits the first-draft combined image into its two original halves, unchanged.
from PIL import Image
im=Image.open('chris_fig8_orientation.png'); w,h=im.size; x=3043
im.crop((0,0,x,h)).save('fig8_orientation_seq.png'); im.crop((x,0,w,h)).save('fig8_migkas_maps.png')
```

Figure 6: `fig8_migkas_maps_clean.png` — `fig8_migkas_maps_clean.py`



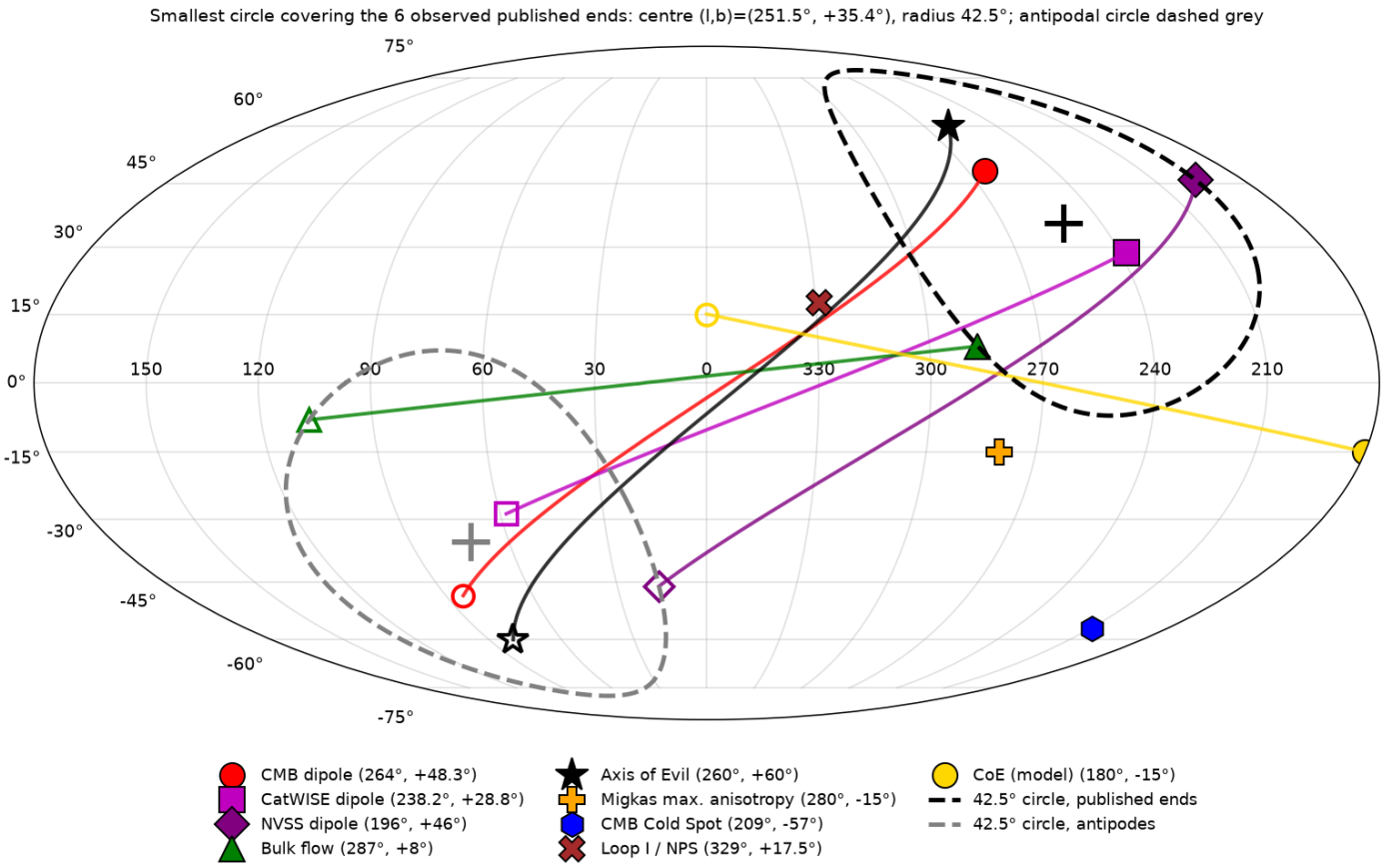
```
# Clean version of fig8_migkas_maps.png (letter figure 5). Removes crop/screenshot blemishes only; data, colours, layout unchanged.
# Source: /workspace/coepdf/fig/Fig06_overlay_one.png (the stack of the three make_migkas_fig8_overlay_matched.py panels).
# Fixes: (1) take the full stack instead of split_fig8.py's x=3043 cut, which pulled in 123 px of the left figure's legend text
# ("(±9°", "of 360°)"); (2) blank the strip under each "Galactic longitude" label, which holds the next sub-panel's
# clipped "-90°" tick, "1.2" colourbar label and red dotted map edge from the cropped Migkas raster; (3) replace the
# half-clipped "Galactic latitude" label with a full one; (4) smooth the saw-tooth HEALPix pixel edge of the Mollweide map
# by whitening only a thin annulus just outside a fitted ellipse.
import numpy as np; from PIL import Image, ImageDraw, ImageFont
SRC='/workspace/coepdf/fig/Fig06_overlay_one.png'; OUT='fig8_migkas_maps_clean.png'
im=Image.open(SRC).convert('RGB'); a=np.asarray(im).copy(); H,W,_=a.shape
OFFS=[0,1061,2121] # vertical offsets of the 3 sub-panels (measured row structure)
INSET=float(__import__('os').environ.get('INSET','4'))
for o in OFFS:
   a[o+930:o+962,:]=255 # stray next-panel fragments under the longitude label
   p=a[o+100:o+930].astype(int); s=p.max(2)-p.min(2); m=s>60; m[:,1450:]=False
   # least-squares ellipse through left/right edge points of each row (tooth noise averages out)
   P=[]
   for y in range(m.shape[0]):
      xs=np.where(m[y])[0]
      if len(xs)>50: P+= [(xs.min(),y+o+100),(xs.max(),y+o+100)]
   for x in range(200,1450,4):
      ys=np.where(m[:,x])[0]
      if len(ys)>50: P+= [(x,ys.min()+o+100),(x,ys.max()+o+100)]
   P=np.array(P,float); X,Y=P[:,0],P[:,1]
   # axis-aligned fit: x^2/Rx^2... solve a x^2 + b y^2 + d x + e y = 1
   c=np.linalg.lstsq(np.c_[X**2,Y**2,X,Y],np.ones_like(X),rcond=None)[0]
   cx=-c[2]/(2*c[0]); cy=-c[3]/(2*c[1]); k=1+c[0]*cx**2+c[1]*cy**2
   Rx=np.sqrt(k/c[0])-INSET; Ry=np.sqrt(k/c[1])-INSET; print('ellipse',o,round(cx,1),round(cy,1),round(Rx,1),round(Ry,1))
   Yg,Xg=np.mgrid[o+100:o+930,0:1450]; r=np.sqrt(((Xg-cx)/Rx)**2+((Yg-cy)/Ry)**2)
   sub=a[o+100:o+930,0:1450]; m2=(s>12)[:,:1450]; sub[(r>1.0)&(r<1.06)&m2]=255 # whiten only coloured map pixels just outside the ellipse (labels untouched)
   a[o+360:o+612,150:200]=255 # half-clipped latitude label
im=Image.fromarray(a)
try: F=ImageFont.truetype('DejaVuSans-Bold.ttf',30)
except Exception: F=ImageFont.load_default()
for o in OFFS:
   t=Image.new('RGBA',(300,40),(255,255,255,0)); ImageDraw.Draw(t).text((150,20),'Galactic latitude',font=F,fill='black',anchor='mm')
   t=t.rotate(90,expand=True); im.paste(t,(158,o+486-150),t)
im.save(OUT); print(OUT,im.size)
```

Figure 7: `chris_AB_stacked.png` — `stack_AB.py`



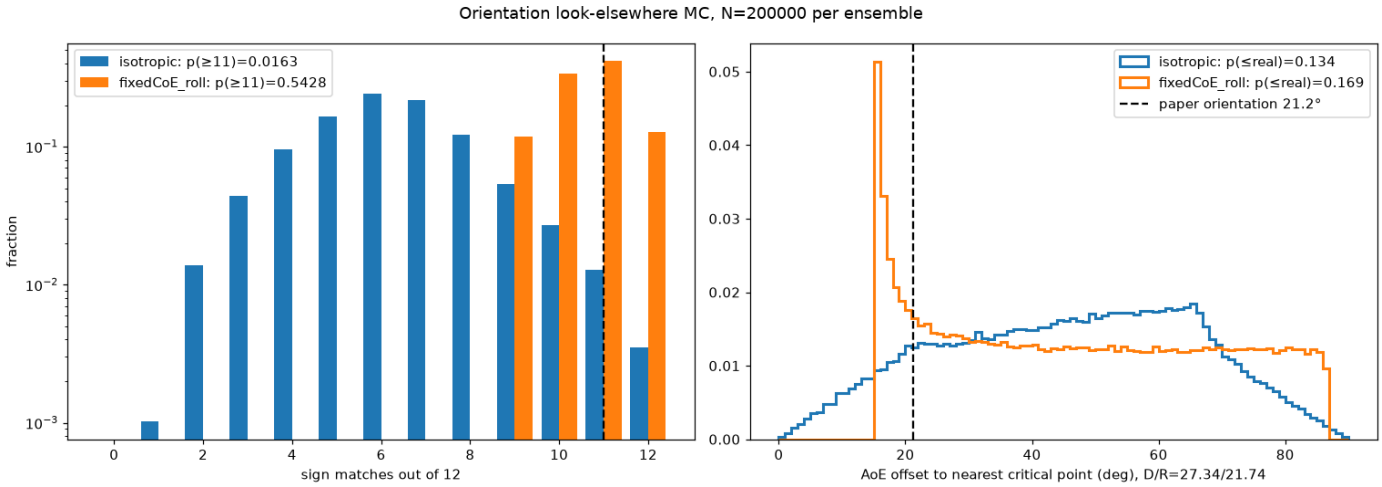
```
# Stacks the first-draft A|B image vertically (A on top), with no other changes.
from PIL import Image; import numpy as np
im=Image.open('chris_AB_residual_spots.png'); w,h=im.size
a=np.asarray(im.convert('RGBA')); ink=((a[...,:3].min(2)<245)&(a[...,3]>0)).sum(0)
c=w//2; x=min(range(int(w*.4),int(w*.6)),key=lambda i:(ink[i],abs(i-c)))
A=im.crop((0,0,x,h)); B=im.crop((x,0,w,h))
out=Image.new('RGBA',(max(A.width,B.width),2*h),(255,255,255,0)); out.paste(A,(0,0)); out.paste(B,(0,h)); out.save('chris_AB_stacked.png'); print(x,ink[x])
```

Figure 8: **dipoles_sky_map.png — dipoles_sky_map.py**



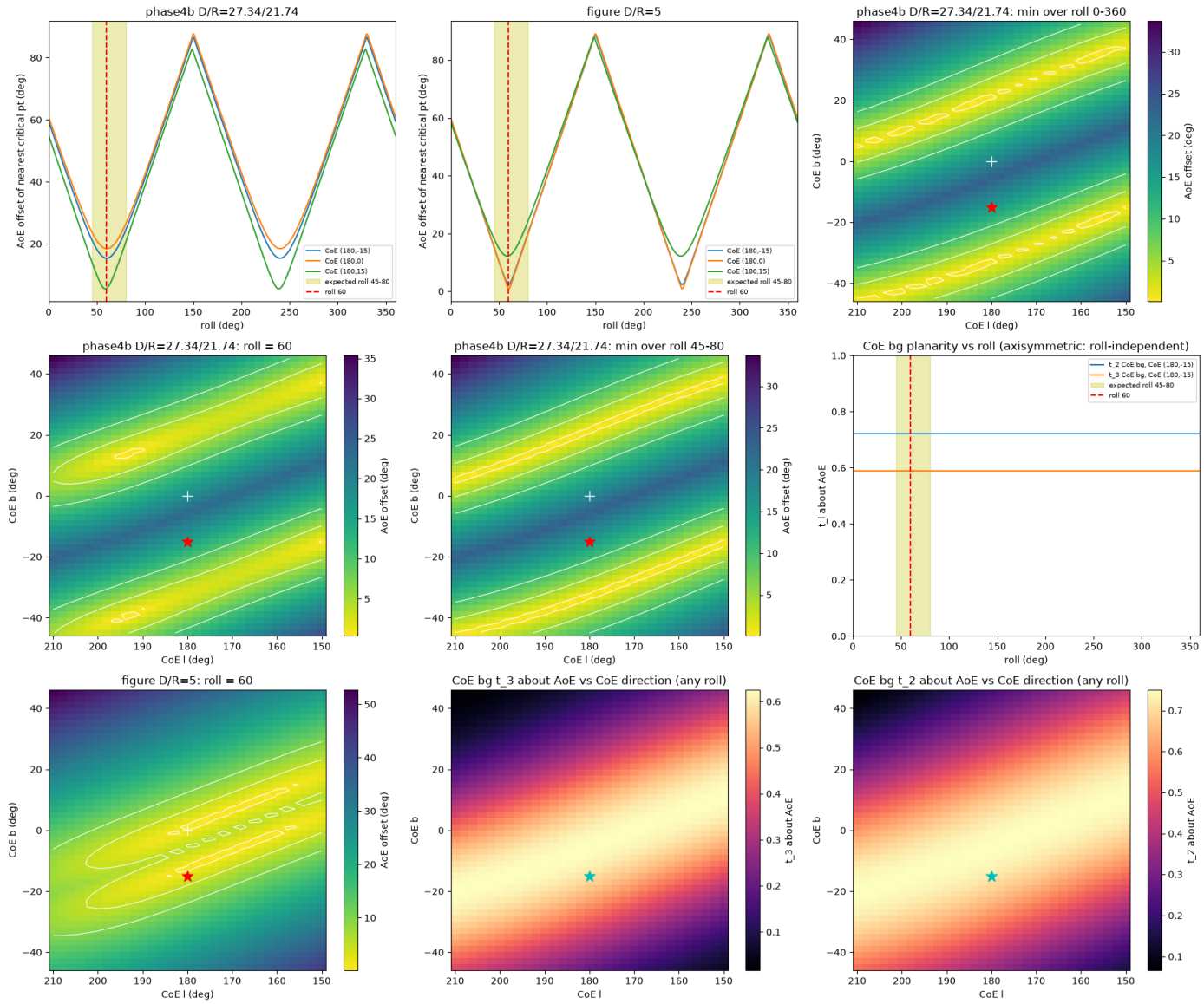
```
# Galactic Mollweide map of the published dipoles/axes and anomaly directions, with the CoE direction.
import os,numpy as np,matplotlib;matplotlib.use('Agg');import matplotlib.pyplot as plt
H=os.path.dirname(os.path.abspath(__file__))
P=[('CMB dipole',264.0,48.3,'r','o',True),('CatWISE dipole',238.2,28.8,'m','s',True),('NVSS dipole',196,46,'purple','D',True),
('Bulk flow',287,8,'g','^',True),('Hemispherical asym.',230,-20,'c','v',True),('Axis of Evil',260,60,'k','*',True),
('Migkas max. anisotropy',280,-15,'orange','P',False),('CMB Cold Spot',209,-57,'b','h',False),('Loop I / NPS',329,17.5,'brown','X',False),('CoE (model)
      ',180,-15,'gold','o',True)]
def xy(l,b):return -np.radians(((l+180)%360)-180),np.radians(b)
fig=plt.figure(figsize=(12,6.6));ax=fig.add_subplot(111,projection='mollweide');ax.grid(alpha=.4)
for n,l,b,c,mk,anti in P:
  x,y=xy(l,b);ax.plot(x,y,mk,color=c,mec='k',ms=14 if mk!='*' else 20,label=f'{n} ({l:g}°, {b:+g}°)')
  if anti:x2,y2=xy(l+180,-b);ax.plot(x2,y2,mk,mfc='none',mec=c,mew=2,ms=12 if mk!='*' else 18)
ax.set_xticks(np.radians([-150,-120,-90,-60,-30,0,30,60,90,120,150]));ax.set_xticklabels(['150','120','90','60','30','0','330','300','270','240','210'])
ax.legend(loc='upper center',bbox_to_anchor=(.5,-.05),ncol=3,fontsize=9,frameon=False)
ax.set_title('Galactic Mollweide, l=0 at centre, increasing to the left. Filled: published direction; open: antipode',fontsize=10,pad=14)
fig.tight_layout();fig.savefig(H+'/dipoles_sky_map.png',dpi=140,facecolor='white',bbox_inches='tight')
```

Figure 9: `lee_hist.png` — `lee_hist.py`



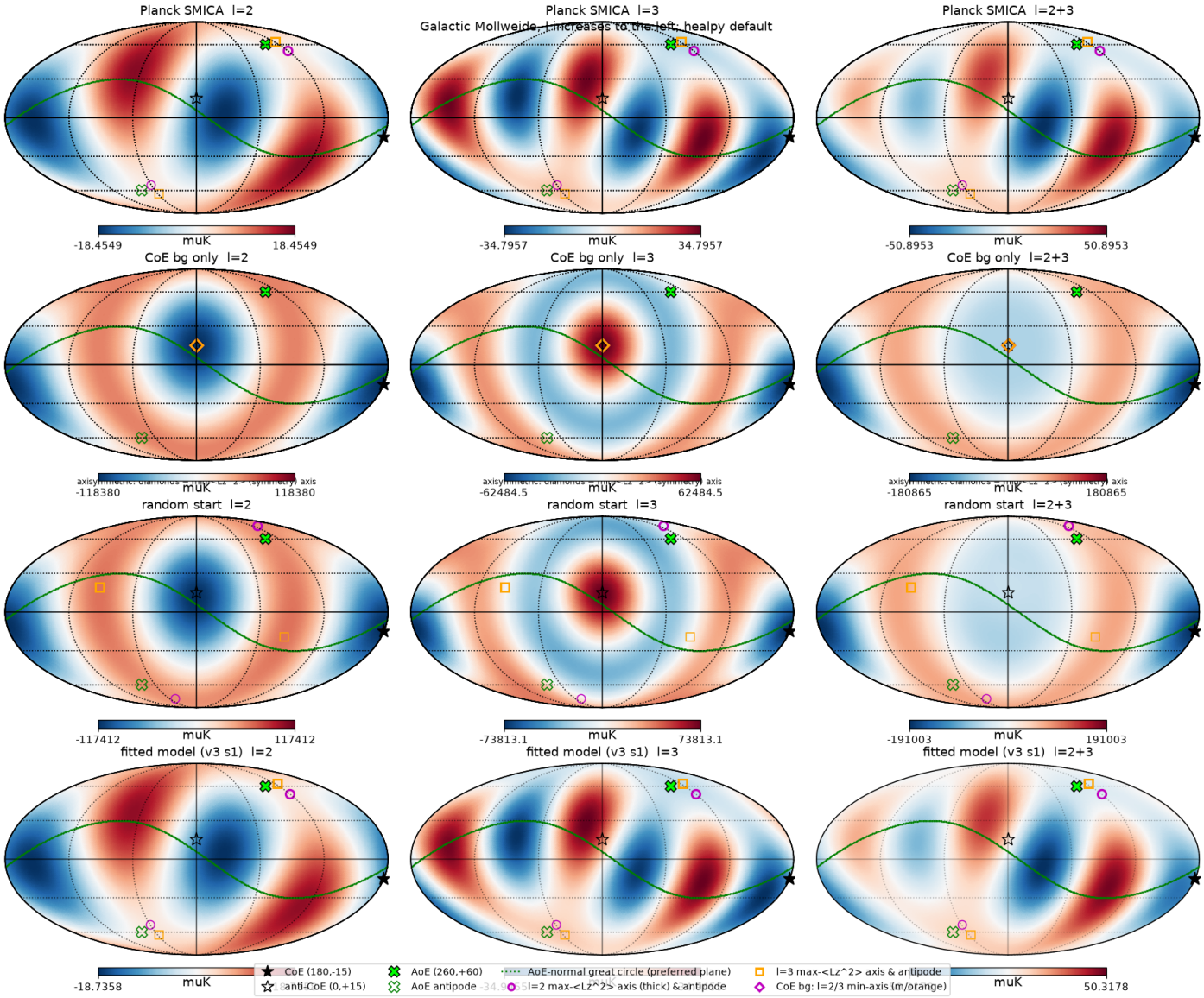
```
import json,numpy as np,matplotlib;matplotlib.use('Agg');import matplotlib.pyplot as plt
H=__file__.rsplit('/',1)[0];d=json.load(open(H+'/lee_results.json'));r=d['real']
fig,ax=plt.subplots(1,2,figsize=(14,5))
for tag,c in [('isotropic','C0'),('fixedCoE_roll','C1')]:
 h=np.array(d['mc'][tag]['score_hist']);ax[0].bar(np.arange(13)+(-.2 if c=='C0' else .2),h/h.sum(),.4,color=c,label=f"{tag}: p≥({r['score12']})={d['mc'][tag]['p_score>=real']:.4f}")
 o=np.load(H+f'/off_{tag}.npy');ax[1].hist(o,bins=90,range=(0,90),density=True,histtype='step',lw=2,color=c,label=f"{tag}: p≤(real)={d['mc'][tag]['aoe']['phase4b joint D=27.34/R_ls=21.74']['p_aoe<=real']:.3f}")
ax[0].axvline(r['score12'],color='k',ls='--');ax[0].set_xlabel('sign matches out of 12');ax[0].set_ylabel('fraction');ax[0].legend();ax[0].set_yscale('log')
o0=d['mc']['isotropic']['aoe']['phase4b joint D=27.34/R_ls=21.74']['real_offset_deg']
ax[1].axvline(o0,color='k',ls='--',label=f'paper orientation {o0:.1f}°');ax[1].set_xlabel('AoE offset to nearest critical point (deg), D/R=27.34/21.74');ax[1].legend()
fig.suptitle(f"Orientation look-elsewhere MC, N={d['N']} per ensemble");fig.tight_layout();fig.savefig(H+'/lee_hist.png',dpi=110)
```

Figure 10: `fig4_orientation_scan.png` — `fig4_orient_scan.py`



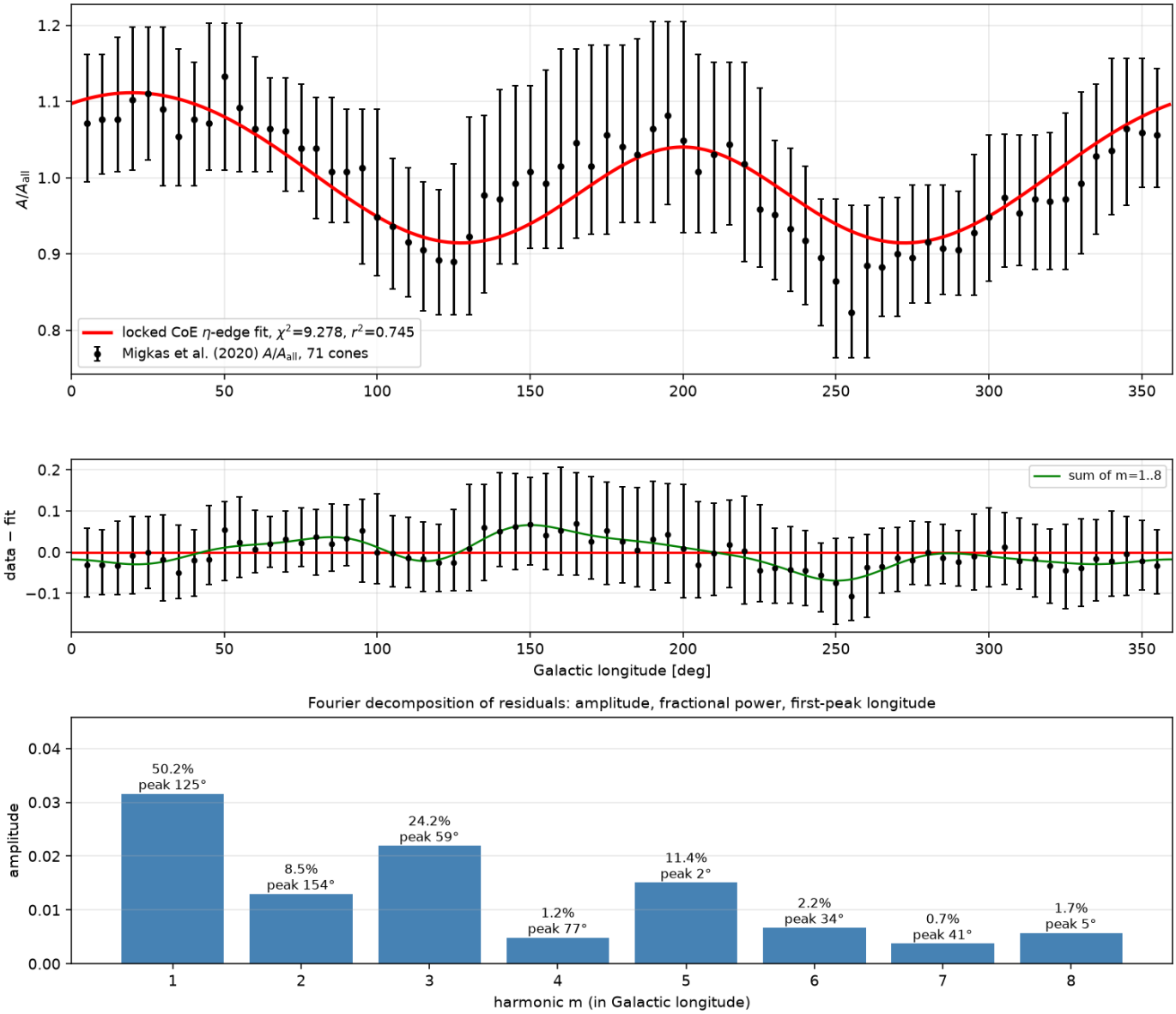
```
# Orientation scan. Convention = orient_lee/lee.py: R = rot(coe,roll) @ Rz(lc-180) @ rot(y,bc); model CoE axis (180,0) -> (lc,bc).
# Critical pts: model-frame b=0, l=180+-theta, theta=arccos(R/2D), plus antipodes. Offset = min angle to AoE.
# B planarity: B is axisymmetric about CoE (min-eig ratio <1e-11), so t_l about AoE depends only on angle(CoE,AoE); evaluated by measuring t_l of B about axes at that angle from its own axis. Roll does not enter.
import sys,json;sys.path.insert(0,'.');from common import *
import matplotlib;matplotlib.use('Agg');import matplotlib.pyplot as plt
exec(open('/workspace/coepdf/joint_solve/phase5/orient_lee/lee.py').read().split("if __name__")[0].split("MM=json")[0])
def vec(l,b):l,b=np.radians(l),np.radians(b);return np.stack([np.cos(b)*np.cos(l),np.cos(b)*np.sin(l),np.sin(b)],-1)
def rot(ax,a):
 ax=ax/np.linalg.norm(ax);a=np.radians(a);K=np.array([[0,-ax[2],ax[1]],[ax[2],0,-ax[0]],[-ax[1],ax[0],0]]);return np.eye(3)+np.sin(a)*K+(1-np.cos(a))*K@K
A=vec(260.,60.);Z=np.array([0,0,1.]);Y=np.array([0,1.,0])
def off(lc,bc,roll,DR):
 R1=rot(Z,lc-180)@rot(Y,bc);c=R1@vec(180,0);Rm=rot(c,roll)@R1
 th=np.degrees(np.arccos(1/(2*DR)));P=vec(np.array([180+th,180-th]),np.zeros(2))@Rm.T
 return float(np.degrees(np.arccos(min(1,np.abs(P@A).max()))))
DRS={'phase4b D/R=27.34/21.74':27.34/21.74,'figure D/R=5':5.0}
rolls=np.arange(0,360.1,1.);LC=np.arange(150,210.1,2);BC=np.arange(-45,45.1,2)
assert abs(np.degrees(np.arccos(rot(Y,-15)@vec(180,0)@vec(180,-15)))) < 1e-6
# B planarity vs angle to AoE
alB=ALM['CoE_bg_only'];perp=np.cross(nco,Z);perp/=np.linalg.norm(perp);angs=np.arange(0,180.1,2)
g={l:[] for l in (2,3)}
for a in angs:
 n=np.cos(np.radians(a))*nco+np.sin(np.radians(a))*perp;ax=lb(n) if True else None
 lo,la=hp.vec2ang(n,lonlat=True)
 for l in (2,3):g[l].append(tl_about(alB,(float(lo[0]),float(la[0])),l)[0])
def tB(lc,bc,l):a=np.degrees(np.arccos(np.clip(vec(lc,bc)@A,-1,1)));return np.interp(a,angs,g[l])
out={'convention':'lee.py; paper orientation = (180,-15), roll 45','DR':DRS}
fig,ax=plt.subplots(3,3,figsize=(18,15));ax=ax.ravel()
def band(a):a.axvspan(45,80,color='y',alpha=.3,label='expected roll 45-80');a.axvline(60,color='r',ls='--',label='roll 60')
for i,(nm,DR) in enumerate(DRS.items()):
 a=ax[i]
 for (lc,bc) in [(180,-15),(180,0),(180,15)]:
  o=[off(lc,bc,r,DR) for r in rolls];a.plot(rolls,o,label=f'CoE ({lc},{bc})');out[f'{nm} offset vs roll CoE({lc},{bc})']={'roll60':o[60],'roll300':o[300],'min_all':[min(o),float(rolls[int(np.argmin(o))])],'min_45_80':[min(o[45:81]),float(rolls[45+int(np.argmin(o[45:81]))])]}
 band(a);a.set_xlabel('roll (deg)');a.set_ylabel('AoE offset of nearest critical pt (deg)');a.set_title(nm);a.set_xlim(0,360);a.legend(fontsize=7)
 for j,(ttl,fn) in enumerate([('roll = 60',lambda lc,bc:off(lc,bc,60,DR)),('min over roll 45-80',lambda lc,bc:min(off(lc,bc,r,DR) for r in range(45,81))),('min over roll 0-360',lambda lc,bc:min(off(lc,bc,r,DR) for r in range(0,360,2)))]):
  if i==1 and j>=1:continue # slot ax[7] is redrawn as the t_3 map below; skip so no stray colorbar
  G=np.array([[fn(lc,bc) for lc in LC] for bc in BC]);a=ax[3+3*i+j] if j<2 else ax[2]
  im=a.pcolormesh(LC,BC,G,shading='nearest',cmap='viridis_r');plt.colorbar(im,ax=a,label='AoE offset (deg)')
  a.contour(LC,BC,G,[1,5,10],colors='w',linewidths=.8);a.plot(180,-15,'r*',ms=12);a.plot(180,0,'w+',ms=12)
  a.set_xlabel('CoE l (deg)');a.set_ylabel('CoE b (deg)');a.set_title(f'{nm}: {ttl}');a.invert_xaxis()
  k=np.unravel_index(np.argmin(G),G.shape);out[f'{nm} map {ttl}']={'min':float(G[k]),'at_lc_bc':[float(LC[k[1]]),float(BC[k[0]])],'at(180,-15)':float(G[list(BC).index(-15),list(LC).index(180)]),'at(180,-15)':float(G[list(BC).index(-15),list(LC).index(180)])}
# B planarity panels
a=ax[5];a.cla()
for l in (2,3):a.plot(rolls,[tB(180,-15,l)]*len(rolls),label=f't_{l} CoE bg, CoE (180,-15)')
band(a);a.set_xlim(0,360);a.set_ylim(0,1);a.set_xlabel('roll (deg)');a.set_ylabel('t_l about AoE');a.set_title('CoE bg planarity vs roll (axisymmetric: roll-independent)');a.legend(fontsize=7)
for l,a in [(2,ax[8]),(3,ax[7])]:
 a.cla();G=np.array([[tB(lc,bc,l) for lc in LC] for bc in BC]);im=a.pcolormesh(LC,BC,G,shading='nearest',cmap='magma');plt.colorbar(im,ax=a,label=f't_{l} about AoE')
 a.plot(180,-15,'c*',ms=12);a.set_xlabel('CoE l');a.set_ylabel('CoE b');a.set_title(f'CoE bg t_{l} about AoE vs CoE direction (any roll)');a.invert_xaxis()
 out[f'B t_{l} vs CoE dir']={'at(180,-15)':float(tB(180,-15,l)),'max_in_grid':float(G.max()),'min_in_grid':float(G.min())}
out['B t_l vs angle(CoE,AoE)']={str(l):dict(zip(map(str,angs[::5]),np.round(g[l][::5],3).tolist())) for l in (2,3)}
plt.tight_layout();plt.savefig('fig4_orientation_scan.png',dpi=100);json.dump(out,open('fig4_values.json','w'),indent=1);print(json.dumps(out,indent=1))
```

Figure 11: `fig1_lowl_maps.png` — `fig1_maps.py`



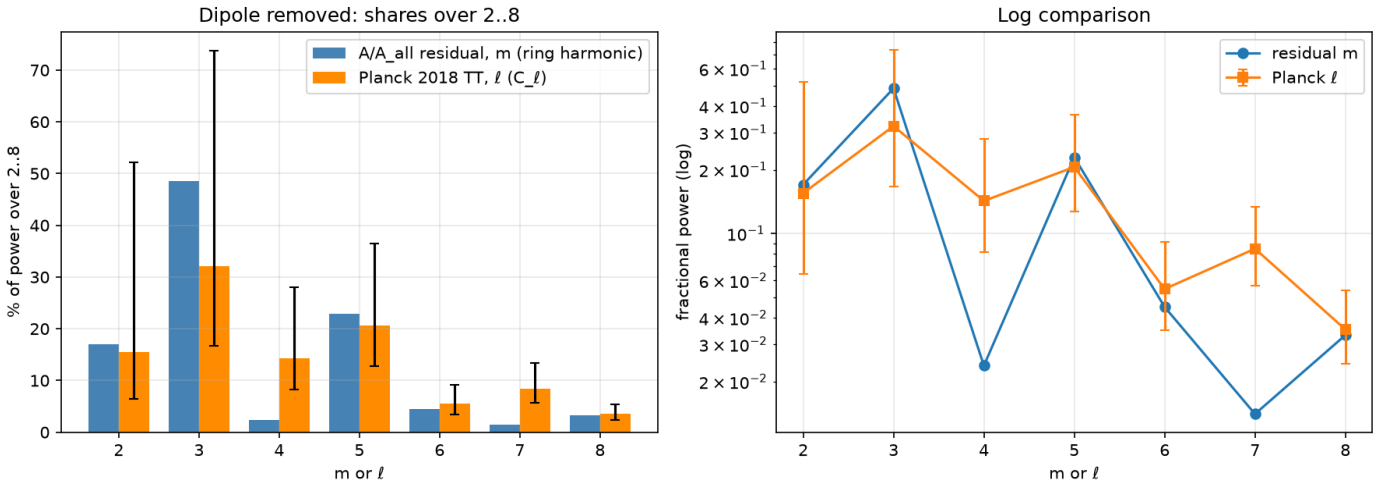
```
import sys;sys.path.insert(0,'.');from common import *
import matplotlib;matplotlib.use('Agg');import matplotlib.pyplot as plt
def P(l,b,**k):hp.projplot(l,b,lonlat=True,**k)
fig=plt.figure(figsize=(16,13))
for i,k in enumerate(ORDER):
   a=ALM[k]
   for j,(ls,t) in enumerate([([2],'l=2'),([3],'l=3'),([2,3],'l=2+3')]):
      f=np.zeros(hp.Alm.getsize(LM),complex)
      for l in ls:
         for m in range(l+1):f[hp.Alm.getidx(LM,l,m)]=a[hp.Alm.getidx(LM,l,m)]
      mp=hp.alm2map(f,64,lmax=LM)*T0;v=np.abs(mp).max()
      hp.mollview(mp,fig=fig.number,sub=(4,3,3*i+j+1),cmap='RdBu_r',min=-v,max=v,unit='muK',title=f'{LAB[k]} {t}',notext=True)
      hp.graticule(dpar=30,dmer=60,alpha=.3)
      P(*COE,marker='*',ms=13,color='k',ls='');P(0,15,marker='*',ms=13,mfc='none',mec='k',ls='')
      P(260,60,marker='X',ms=11,color='lime',mec='k',ls='');P(80,-60,marker='X',ms=11,mfc='none',mec='g',ls='')
      # AoE-normal great circle
      n=np.array(hp.ang2vec(260,60,lonlat=True));u=np.cross(n,[0,0,1.]);u/=np.linalg.norm(u);w=np.cross(n,u);ph=np.linspace(0,2*np.pi,400)
      c=np.outer(np.cos(ph),u)+np.outer(np.sin(ph),w);lo,la=hp.vec2ang(c,lonlat=True);P(lo,la,color='g',ls='',marker='.',ms=1.5)
      for l,mk,cl in [(2,'o','m'),(3,'s','orange')]:
         ev,e=eig(a,l)
         if k=='CoE_bg_only':
            x=lb(e[:,0]);P(*x,marker='D',ms=8,mfc='none',mec=cl,ls='',mew=2)
         else:
            x=lb(e[:,-1]);P(*x,marker=mk,ms=8,mfc='none',mec=cl,mew=2,ls='');P(x[0]+180,-x[1],marker=mk,ms=8,mfc='none',mec=cl,mew=1,ls='')
      if k=='CoE_bg_only':plt.text(0,-1.25,'axisymmetric: diamonds = min-<Lz^2> (symmetry) axis',ha='center',fontsize=8)
from matplotlib.lines import Line2D as L2
h=[L2([],[],marker='*',color='k',ls='',ms=12,label='CoE (180,-15)'),L2([],[],marker='*',mfc='none',mec='k',ls='',ms=12,label='anti-CoE (0,+15)'),
 L2([],[],marker='X',color='lime',mec='k',ls='',ms=10,label='AoE (260,+60)'),L2([],[],marker='X',mfc='none',mec='g',ls='',ms=10,label='AoE antipode'),
 L2([],[],color='g',ls=':',label='AoE-normal great circle (preferred plane)'),L2([],[],marker='o',mfc='none',mec='m',ls='',mew=2,label='l=2 max-<Lz^2> axis
       (thick) & antipode'),
 L2([],[],marker='s',mfc='none',mec='orange',ls='',mew=2,label='l=3 max-<Lz^2> axis & antipode'),L2([],[],marker='D',mfc='none',mec='m',ls='',mew=2,label='
       CoE bg: l=2/3 min-axis (m/orange)')]
fig.legend(handles=h,loc='lower center',ncol=4,fontsize=9);fig.suptitle('Galactic Mollweide, l increases to the left; healpy default')
plt.savefig('fig1_lowl_maps.png',dpi=110,bbox_inches='tight')
```

Figure 12: **A_fit_residual_fourier.png — A_fit_residual_fourier.py**



```
# A/A_all data + locked CoE eta-edge fit, residuals, and Fourier decomposition of the residuals in l.
# Same data and model files as Fig04_amp_full.py.
import os,json,numpy as np,matplotlib;matplotlib.use('Agg');import matplotlib.pyplot as plt
H=os.path.dirname(os.path.abspath(__file__))
d=np.genfromtxt('/workspace/cosmology/migkas/fig6_digitized.csv',delimiter=',',names=True);l,y=d['l_deg'],d['A_over_Aall'];el,eh=d['A_err_low'],d['A_err_high'];s=0.5*(el+eh)
c=np.loadtxt('/workspace/cosmology-course/notes/catwise-eta-migkas-fit-d14.3.csv',delimiter=',',skiprows=1)
m=np.interp(l,c[:,0],c[:,1],period=360);r=y-m
chi2=float(np.sum(r**2/s**2));r2=float(1-np.sum(r**2)/np.sum((y-y.mean())**2))
# least-squares harmonic fit (points are 5 deg apart but not necessarily full-circle uniform)
M=8;L=np.radians(l);cols=[np.ones_like(L)]
for k in range(1,M+1):cols+=[np.cos(k*L),np.sin(k*L)]
X=np.stack(cols,1);coef=np.linalg.lstsq(X,r,rcond=None)[0]
a=coef[1::2];b=coef[2::2];amp=np.hypot(a,b);ph=np.degrees(np.arctan2(b,a))%360 # r ~ amp*cos(k(l-ph/k))
pw=amp**2/2;frac=pw/np.sum(pw);peak=(ph/np.arange(1,M+1))%(360/np.arange(1,M+1))
fig,ax=plt.subplots(3,1,figsize=(11,9.5),gridspec_kw=dict(height_ratios=[3,1.4,2]))
ax[0].errorbar(l,y,[el,eh],fmt='ko',ms=3.5,capsize=2,label='Migkas et al. (2020) $A/A_{\\rm all}$, 71 cones')
ax[0].plot(c[:,0],c[:,1],'r',lw=2.2,label=f'locked CoE $\\eta$-edge fit, $\\chi^2$={chi2:.3f}, $r^2$={r2:.3f}')
ax[0].set_ylabel('$A/A_{\\rm all}$');ax[0].legend(fontsize=9);ax[0].grid(alpha=.3);ax[0].set_xlim(0,360)
ax[1].errorbar(l,r,[el,eh],fmt='ko',ms=3,capsize=2);ax[1].axhline(0,color='r')
lf=np.linspace(0,360,721);ax[1].plot(lf,np.stack([np.ones_like(lf)]+sum([[np.cos(k*np.radians(lf)),np.sin(k*np.radians(lf))] for k in range(1,M+1)],[]),1)@coef,'g',lw=1.3,label=f'sum of m=1..{M}')
ax[1].set_ylabel('data −fit');ax[1].set_xlabel('Galactic longitude [deg]');ax[1].set_xlim(0,360);ax[1].grid(alpha=.3);ax[1].legend(fontsize=8)
k=np.arange(1,M+1);ax[2].bar(k,amp,color='steelblue')
for i in range(M):ax[2].text(k[i],amp[i],f'{100*frac[i]:.1f}%\npeak {peak[i]:.0f}°',ha='center',va='bottom',fontsize=8.5)
ax[2].set_xlabel('harmonic m (in Galactic longitude)');ax[2].set_ylabel('amplitude');ax[2].set_xticks(k);ax[2].set_ylim(0,amp.max()*1.45);ax[2].grid(alpha=.3,axis='y')
ax[2].set_title('Fourier decomposition of residuals: amplitude, fractional power, first-peak longitude',fontsize=10)
fig.tight_layout();fig.savefig(H+'/A_fit_residual_fourier.png',dpi=140,facecolor='white')
out=dict(chi2=chi2,r2=r2,offset=float(coef[0]),modes=[dict(m=int(k[i]),amp=float(amp[i]),phase_deg=float(ph[i]),first_peak_l=float(peak[i]),frac_power=float(frac[i])) for i in range(M)])
json.dump(out,open(H+'/A_fit_residual_fourier.json','w'),indent=1);print(json.dumps(out,indent=0))
```

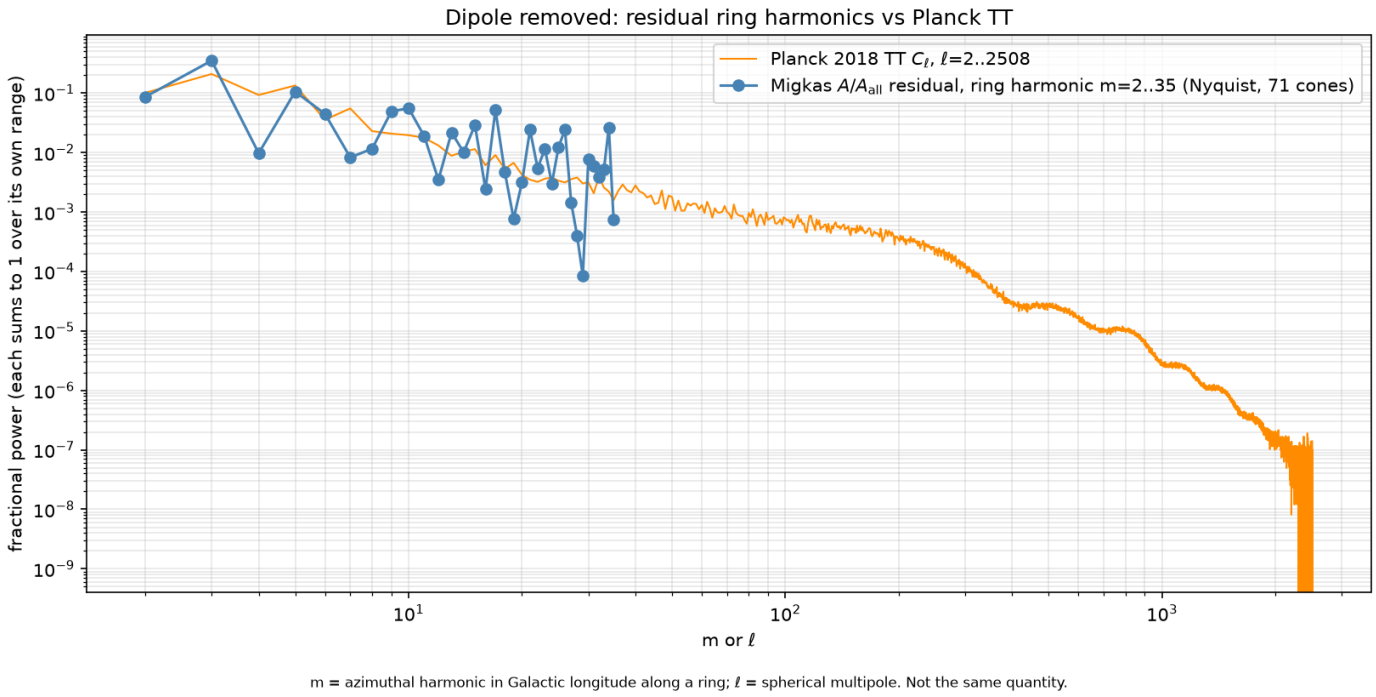
Figure 13: **A_residual_power_vs_planck_noDipole.png — A_residual_power_vs_planck_no**



m = azimuthal harmonic in Galactic longitude along a ring; ℓ = spherical multipole. Not the same quantity.

```
# l=1 / m=1 removed (CMB dipole treated as our peculiar motion). Fractional power renormalized over 2..8.
import json,numpy as np,matplotlib;matplotlib.use('Agg');import matplotlib.pyplot as plt
M=json.load(open('A_fit_residual_fourier.json'))['modes'];m=np.array([x['m'] for x in M]);a2=np.array([x['amp']**2 for x in M])
k=m>=2;m=m[k];r=a2[k]/a2[k].sum()
d=np.loadtxt('COM_PowerSpect_CMB-TT-full_R3.01.txt');d=d[(d[:,0]>=2)&(d[:,0]<=8)]
l=d[:,0];f=2*np.pi/(l*(l+1));C=d[:,1]*f;S=C.sum();p=C/S;lo=d[:,2]*f/S;hi=d[:,3]*f/S
fig,ax=plt.subplots(1,2,figsize=(12,4.5));w=.38
ax[0].bar(m-w/2,100*r,w,label='A/A_all residual, m (ring harmonic)',color='steelblue')
ax[0].bar(l+w/2,100*p,w,yerr=[100*lo,100*hi],capsize=3,label='Planck 2018 TT, □(□C_)',color='darkorange')
ax[0].set(xlabel='m or □',ylabel='% of power over 2..8',title='Dipole removed: shares over 2..8');ax[0].legend()
ax[1].semilogy(m,r,'o-',label='residual m');ax[1].errorbar(l,p,yerr=[np.minimum(lo,p*.99),hi],fmt='s-',capsize=3,label='Planck □')
ax[1].set(xlabel='m or □',ylabel='fractional power (log)',title='Log comparison');ax[1].legend()
for x in ax:x.set_xticks(range(2,9));x.grid(alpha=.3)
fig.text(.5,.005,'m = azimuthal harmonic in Galactic longitude along a ring; □= spherical multipole. Not the same quantity.',ha='center',fontsize=8)
plt.tight_layout(rect=[0,.03,1,1]);plt.savefig('A_residual_power_vs_planck_noDipole.png',dpi=150)
for i in range(7):print(int(m[i]),round(100*r[i],1),int(l[i]),round(100*p[i],1))
```

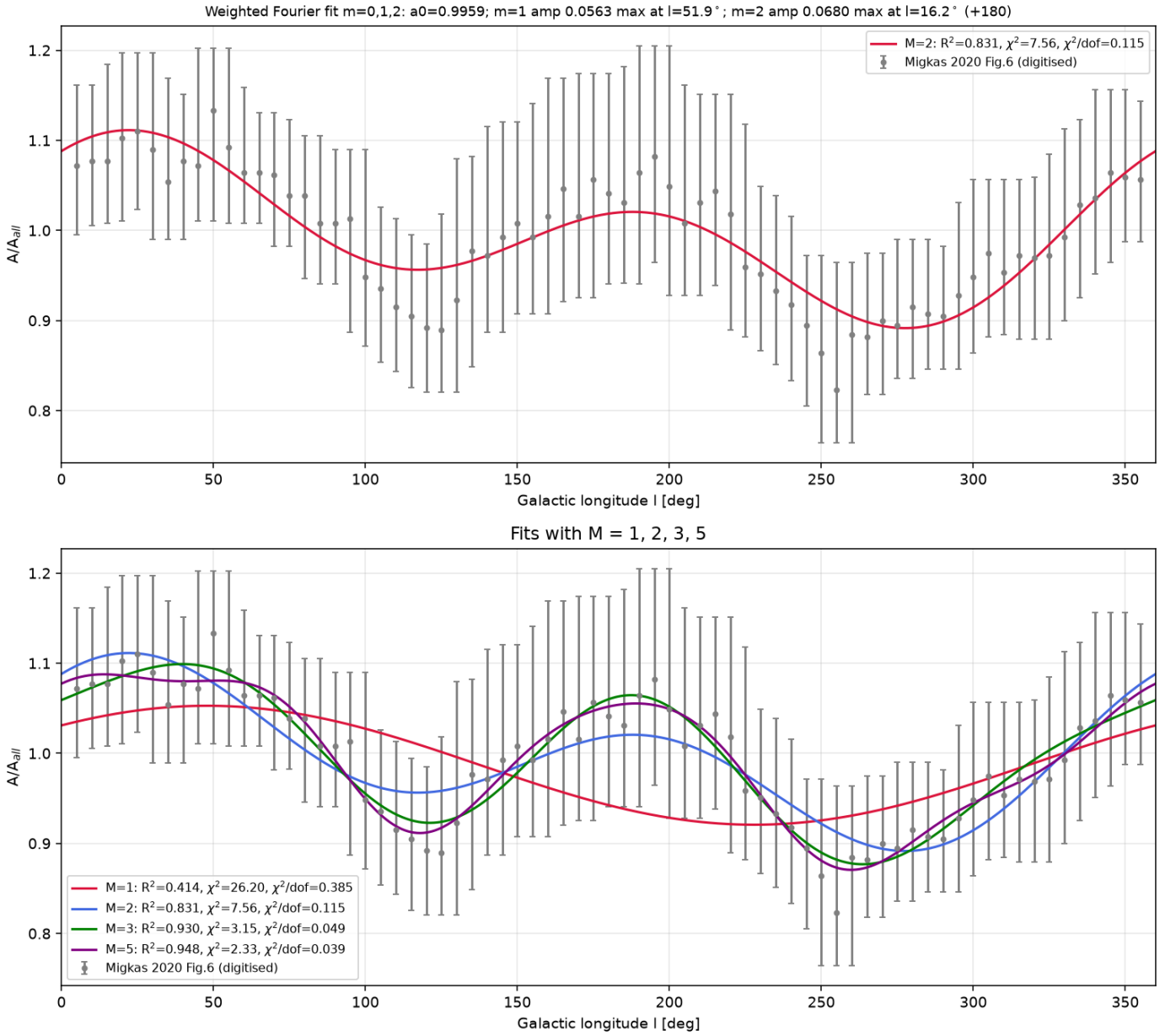
Figure 14: **A_residual_vs_planck_full.png — A_residual_vs_planck_full.py**



m = azimuthal harmonic in Galactic longitude along a ring; ℓ = spherical multipole. Not the same quantity.

```
# Migkas A/A_all residual ring harmonics m=2..35 (Nyquist for 71 cones at 5 deg) vs Planck 2018 TT C_l, l=2..lmax. Dipole (m=1, l=1) removed.
# Each spectrum normalized to sum 1 over its own range (2..35 for m, 2..lmax for l).
import numpy as np,matplotlib;matplotlib.use('Agg');import matplotlib.pyplot as plt
d=np.genfromtxt('/workspace/cosmology/migkas/fig6_digitized.csv',delimiter=',',names=True);l,y=d['l_deg'],d['A_over_Aall']
c=np.loadtxt('/workspace/cosmology-course/notes/catwise-eta-migkas-fit-d14.3.csv',delimiter=',',skiprows=1)
r=y-np.interp(l,c[:,0],c[:,1],period=360);L=np.radians(l);N=len(l);M=(N-1)//2
X=np.stack([np.ones_like(L)]+sum([[np.cos(k*L),np.sin(k*L)] for k in range(1,M+1)],[]),1)
coef=np.linalg.lstsq(X,r,rcond=None)[0];amp2=coef[1::2]**2+coef[2::2]**2
m=np.arange(1,M+1);k=m>=2;pm=amp2[k]/amp2[k].sum()
P=np.loadtxt('COM_PowerSpect_CMB-TT-full_R3.01.txt');ll=P[:,0];C=P[:,1]*2*np.pi/(ll*(ll+1));pl=C/C.sum()
fig,ax=plt.subplots(figsize=(11,5.5))
ax.loglog(ll,pl,'-',color='darkorange',lw=1,label='Planck 2018 TT $C_\\ell$, $\\ell$=2..%d'%ll.max())
ax.loglog(m[k],pm,'o-',color='steelblue',label='Migkas $A/A_{\\rm all}$ residual, ring harmonic m=2..%d (Nyquist, %d cones)'%(M,N))
ax.set(xlabel='m or $\\ell$',ylabel='fractional power (each sums to 1 over its own range)',title='Dipole removed: residual ring harmonics vs Planck TT')
ax.grid(alpha=.3,which='both');ax.legend()
fig.text(.5,.005,'m = azimuthal harmonic in Galactic longitude along a ring; $\\ell$ = spherical multipole. Not the same quantity.',ha='center',fontsize=8)
plt.tight_layout(rect=[0,.03,1,1]);plt.savefig('A_residual_vs_planck_full.png',dpi=150)
print('N',N,'M',M,'lmax',ll.max(),'rank',np.linalg.matrix_rank(X));print(np.round(100*pm,2))
```

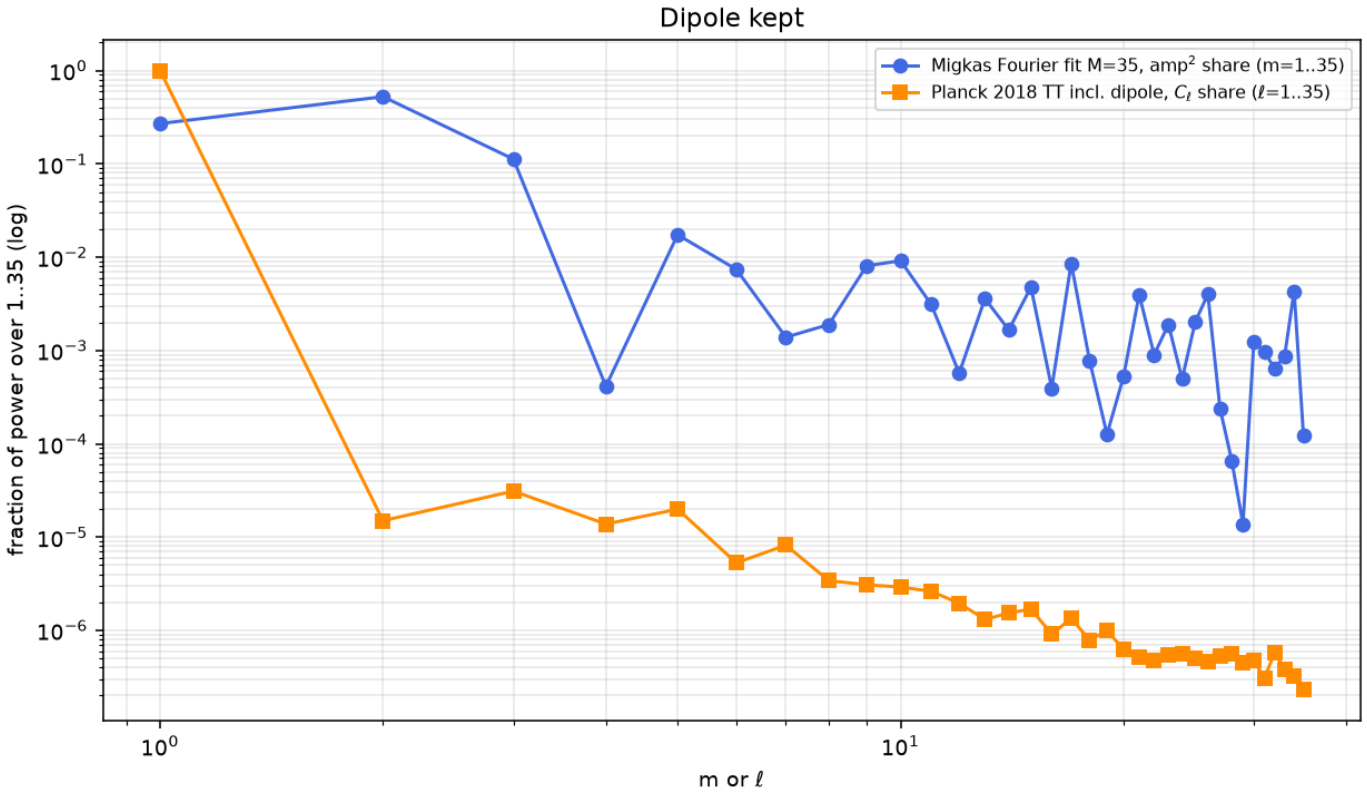
Figure 15: `A_fourier_fits_M1235.png` — `A_fourier_fits_M1235.py`



Weights $1/\sigma^2$, σ=(el+eh)/2. Points come from overlapping 90° regions, so not independent; χ^2/dof treats them as independent.

```
# Naive truncated Fourier fit of Migkas et al. 2020 Fig.6 A/A_all vs Galactic longitude (71 points, 5 deg step).
# Weights 1/sigma^2 with sigma = mean of asymmetric errors (el+eh)/2. Model: a0 + sum_{m=1..M} [a_m cos(m l) + b_m sin(m l)].
# NOTE: the 71 points come from overlapping 90-deg-wide regions (Migkas Sect.5.2), so they are not independent;
# chi2/dof and AIC/BIC below treat them as independent, as a naive fit does.
import numpy as np,matplotlib,json;matplotlib.use('Agg');import matplotlib.pyplot as plt
d=np.genfromtxt('/workspace/cosmology/migkas/fig6_digitized.csv',delimiter=',',names=True);l,y=d['l_deg'],d['A_over_Aall'];el,eh=d['A_err_low'],d['A_err_high']
s=(el+eh)/2;w=1/s**2;L=np.radians(l);n=len(l)
def X(M,LL):return np.stack([np.ones_like(LL)]+sum([[np.cos(k*LL),np.sin(k*LL)] for k in range(1,M+1)],[]),1)
def fit(M):
 A=X(M,L);co=np.linalg.solve(A.T@(A*w[:,None]),A.T@(w*y));r=y-A@co;chi=float((w*r*r).sum())
 ybar=(w*y).sum()/w.sum();R2=1-chi/float((w*(y-ybar)**2).sum());k=2*M+1
 return co,chi,R2,k,chi+2*k,chi+k*np.log(n)
rows=[];res={}
for M in range(1,9):
 co,chi,R2,k,aic,bic=fit(M);res[M]=co;rows.append(dict(M=M,k=k,chi2=chi,dof=n-k,chi2_dof=chi/(n-k),R2w=R2,AIC=aic,BIC=bic))
a0=min(r['AIC'] for r in rows);b0=min(r['BIC'] for r in rows)
for r in rows:r['dAIC']=r['AIC']-a0;r['dBIC']=r['BIC']-b0;print({k:(round(v,3) if isinstance(v,float) else v) for k,v in r.items()})
co=res[2];coef={'a0':float(co[0])}
for m in (1,2):
 a,b=co[2*m-1],co[2*m];amp=np.hypot(a,b);ph=np.degrees(np.arctan2(b,a))%360
 coef['m%d'%m]={'amp':float(amp),'phase_deg_of_max':float(ph/m),'a':float(a),'b':float(b)}
print('M=2',coef);json.dump({'table':rows,'M2_coeffs':coef},open('A_fourier_naive_fit.json','w'),indent=1)
g=np.linspace(0,360,721);G=np.radians(g);fig,ax=plt.subplots(2,1,figsize=(11,10))
for a,Ms in zip(ax,[[2],[1,2,3,5]]):
 a.errorbar(l,y,yerr=[el,eh],fmt='o',color='gray',ms=3,capsize=2,label='Migkas 2020 Fig.6 (digitised)')
 for M,c in zip(Ms,['crimson','royalblue','green','purple'] if len(Ms)>1 else ['crimson']):
  r=rows[M-1];a.plot(g,X(M,G)@res[M],color=c,lw=1.6,label='M=%d: R$^2$=%.3f, $\\chi^2$=%.2f, $\\chi^2$/dof=%.3f'%(M,r['R2w'],r['chi2'],r['chi2_dof']))
 a.set(xlabel='Galactic longitude l [deg]',ylabel='A/A$_{all}$',xlim=(0,360));a.grid(alpha=.3);a.legend(fontsize=8)
c1,c2=coef['m1'],coef['m2']
ax[0].set_title('Weighted Fourier fit m=0,1,2: a0=%.4f; m=1 amp %.4f max at l=%.1f$^\\circ$; m=2 amp %.4f max at l=%.1f$^\\circ$ (+180)'%(coef['a0'],c1['amp'],c1['phase_deg_of_max'],c2['amp'],c2['phase_deg_of_max']),fontsize=9)
ax[1].set_title('Fits with M = 1, 2, 3, 5')
fig.text(.5,.004,'Weights 1/$\\sigma^2$, $\\sigma$=(el+eh)/2. Points come from overlapping 90$^\\circ$ regions, so not independent; $\\chi^2$/dof treats them as independent.',ha='center',fontsize=8)
plt.tight_layout(rect=[0,.015,1,1]);plt.savefig('A_fourier_naive_fit.png',dpi=140)
```

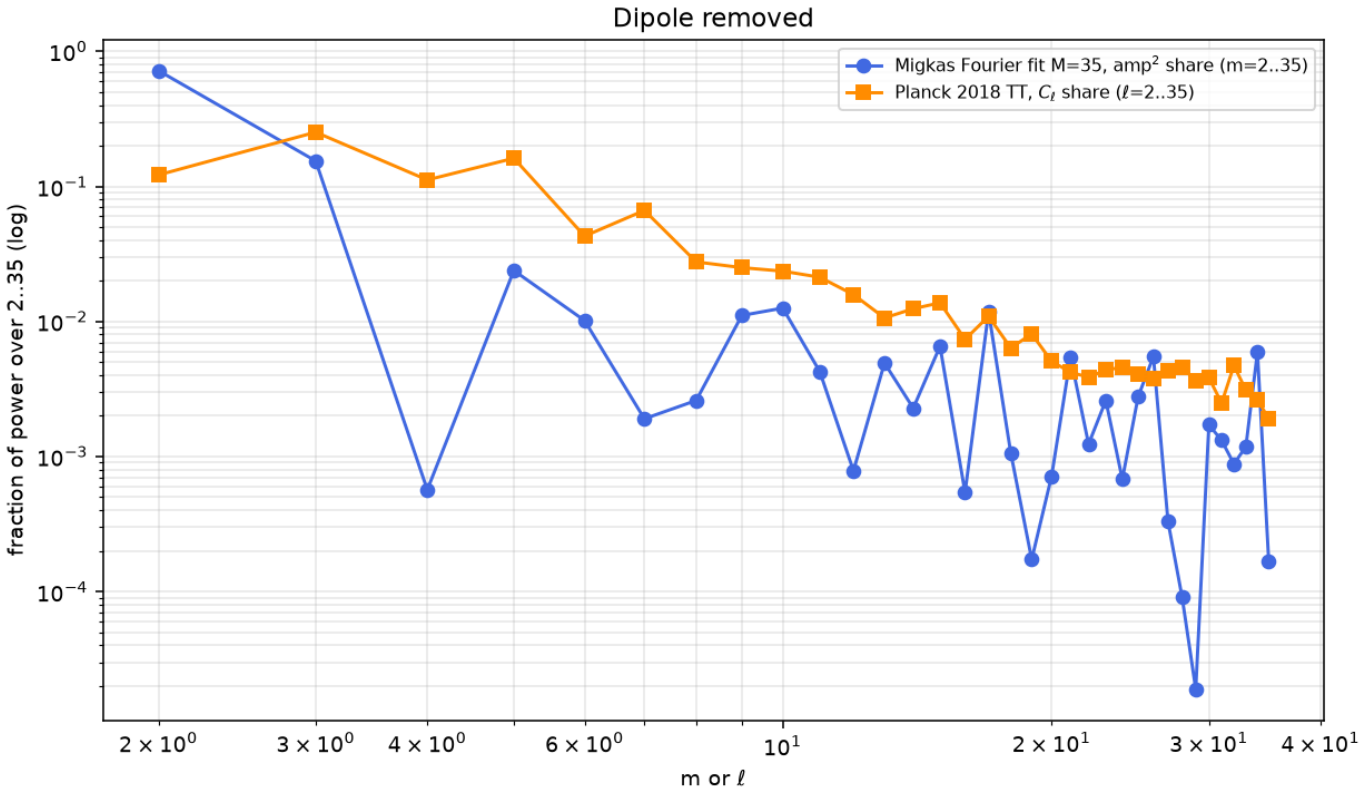
Figure 16: **A_fourier_M35_withDipole_fig.png** — **A_fourier_M35_withDipole_fig.py**



m = azimuthal ring harmonic; ℓ = spherical multipole. Not the same quantity. Planck dipole is largely kinematic.

```
# Power spectrum of weighted Fourier-series fit to Migkas A/A_all (M=35), dipole KEPT; fractions over m=1..8.
# Planck 2018 TT l=2..8 from file plus l=1 dipole A=3362.08 uK (Planck 2018 I, arXiv:1807.06205): C1=4*pi*A^2/9. Fractions over l=1..35.
import numpy as np,matplotlib;matplotlib.use('Agg');import matplotlib.pyplot as plt
d=np.genfromtxt('/workspace/cosmology/migkas/fig6_digitized.csv',delimiter=',',names=True);l,y=d['l_deg'],d['A_over_Aall'];s=.5*(d['A_err_low']+d['A_err_high'])
L=np.radians(l);M=35;X=np.stack([np.ones_like(L)]+sum([[np.cos(k*L),np.sin(k*L)] for k in range(1,M+1)],[]),1)
co=np.linalg.lstsq(X/s[:,None],y/s,rcond=None)[0];a2=co[1::2]**2+co[2::2]**2;m=np.arange(1,M+1);f=a2/a2.sum()
P=np.loadtxt('COM_PowerSpect_CMB-TT-full_R3.01.txt')[:34];ll=np.r_[1,P[:,0]];C=np.r_[4*np.pi*3362.08**2/9,P[:,1]*2*np.pi/(P[:,0]*(P[:,0]+1))];g=C/C.sum()
fig,ax=plt.subplots(figsize=(9,5.5));ax.semilogy(m,f,'o-',color='royalblue',label='Migkas Fourier fit M=35, amp$^2$ share (m=1..35)')
ax.semilogy(ll,g,'s-',color='darkorange',label='Planck 2018 TT incl. dipole, $C_\\ell$ share ($\\ell$=1..35)')
ax.set(xlabel='m or $\\ell$',ylabel='fraction of power over 1..35 (log)',title='Dipole kept');ax.grid(alpha=.3,which='both');ax.legend(fontsize=8);ax.set_xscale('log')
fig.text(.5,.005,'m = azimuthal ring harmonic; $\\ell$ = spherical multipole. Not the same quantity. Planck dipole is largely kinematic.',ha='center',fontsize=8)
plt.tight_layout(rect=[0,.03,1,1]);plt.savefig('A_fourier_fit_power_withDipole_M35.png',dpi=140)
print(np.round(100*f,2));print(np.array2string(100*g,precision=5))
```

Figure 17: **A_fourier_M35_noDipole_fig.png — A_fourier_M35_noDipole_fig.py**



m = azimuthal ring harmonic; ℓ = spherical multipole. Not the same quantity. Weighted by 1/sigma^2.

```
# Weighted Fourier-series fit to Migkas A/A_all, M=35; dipole removed: shares over m=2..35 vs Planck 2018 TT C_l shares over l=2..35.
import numpy as np,matplotlib;matplotlib.use('Agg');import matplotlib.pyplot as plt
d=np.genfromtxt('/workspace/cosmology/migkas/fig6_digitized.csv',delimiter=',',names=True);l,y=d['l_deg'],d['A_over_Aall'];s=.5*(d['A_err_low']+d['A_err_high'])
L=np.radians(l);M=35;X=np.stack([np.ones_like(L)]+sum([[np.cos(k*L),np.sin(k*L)] for k in range(1,M+1)],[]),1)
co=np.linalg.lstsq(X/s[:,None],y/s,rcond=None)[0];a2=(co[1::2]**2+co[2::2]**2)[1:];m=np.arange(2,M+1);f=a2/a2.sum()
P=np.loadtxt('COM_PowerSpect_CMB-TT-full_R3.01.txt')[:34];C=P[:,1]*2*np.pi/(P[:,0]*(P[:,0]+1));g=C/C.sum()
fig,ax=plt.subplots(figsize=(9,5.5));ax.loglog(m,f,'o-',color='royalblue',label='Migkas Fourier fit M=35, amp$^2$ share (m=2..35)')
ax.loglog(P[:,0],g,'s-',color='darkorange',label='Planck 2018 TT, $C_\\ell$ share ($\\ell$=2..35)')
ax.set(xlabel='m or $\\ell$',ylabel='fraction of power over 2..35 (log)',title='Dipole removed');ax.grid(alpha=.3,which='both');ax.legend(fontsize=8)
fig.text(.5,.005,'m = azimuthal ring harmonic; $\\ell$ = spherical multipole. Not the same quantity. Weighted by 1/sigma^2.',ha='center',fontsize=8)
plt.tight_layout(rect=[0,.03,1,1]);plt.savefig('A_fourier_fit_power_noDipole_M35.png',dpi=140)
print(np.round(100*f[:7],1),np.round(100*g[:7],1),round(100*f[7:].sum(),1),round(100*g[7:].sum(),1))
```